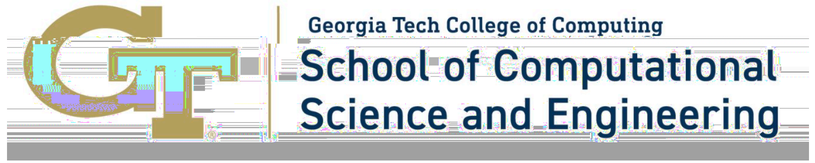
C E 6230: H C A

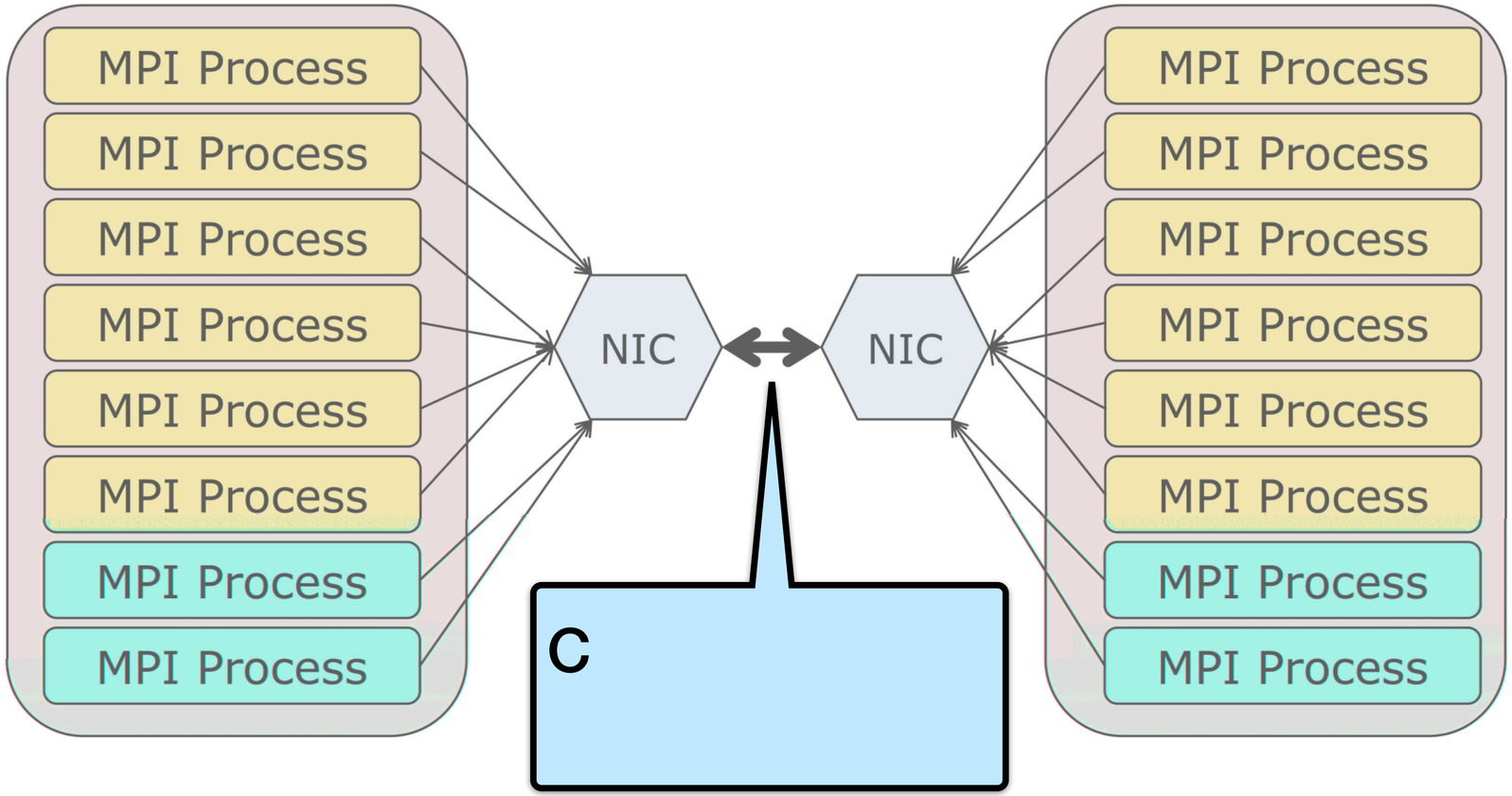
L 10: D - A + C

Helen Xu
hxu615@gatech.edu



D -

Processors are multi-core and many nodes are multi-chip.

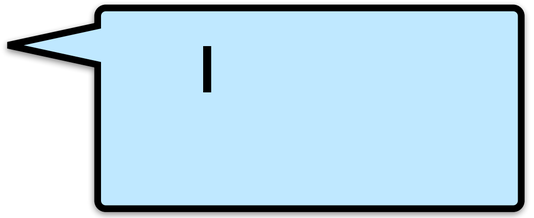


From UC Berkeley CS267

Figure source: Balaji, Gropp, Thakur

L B

D -



Time to send message of length n is roughly:

Time = latency + n * cost_per_word
= latency + n/bandwidth

Time = $\tau + n\mu$

Usually $\tau \gg \mu \gg$ time per flop.

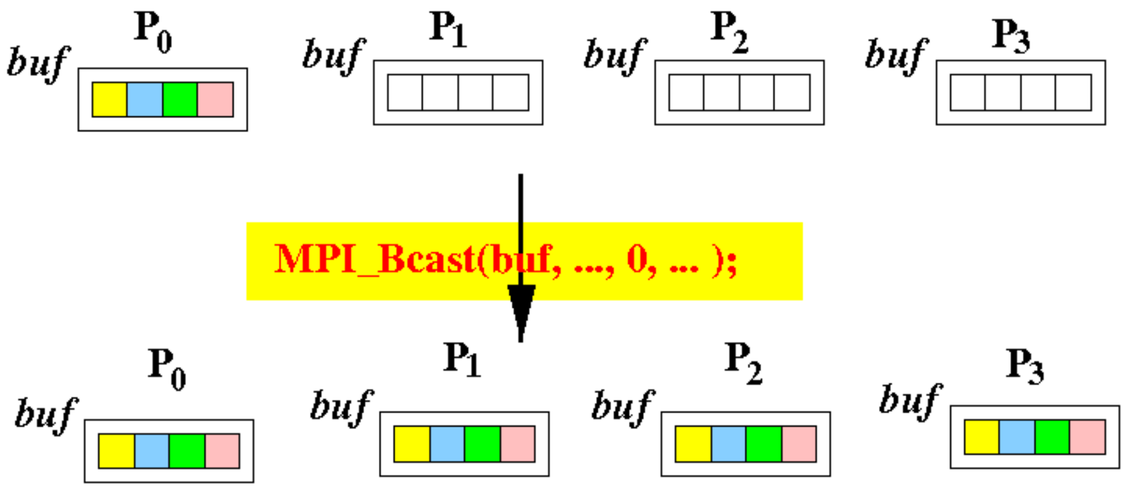
- One long message is cheaper than many short ones.
- Can do hundreds or thousands of flops for cost of one message.
- Sometimes called “alpha-beta” model in the literature where $\alpha = \tau, \beta = \mu$

Lesson: Need **large computation-to-communication ratio** to be efficient.

C C

C

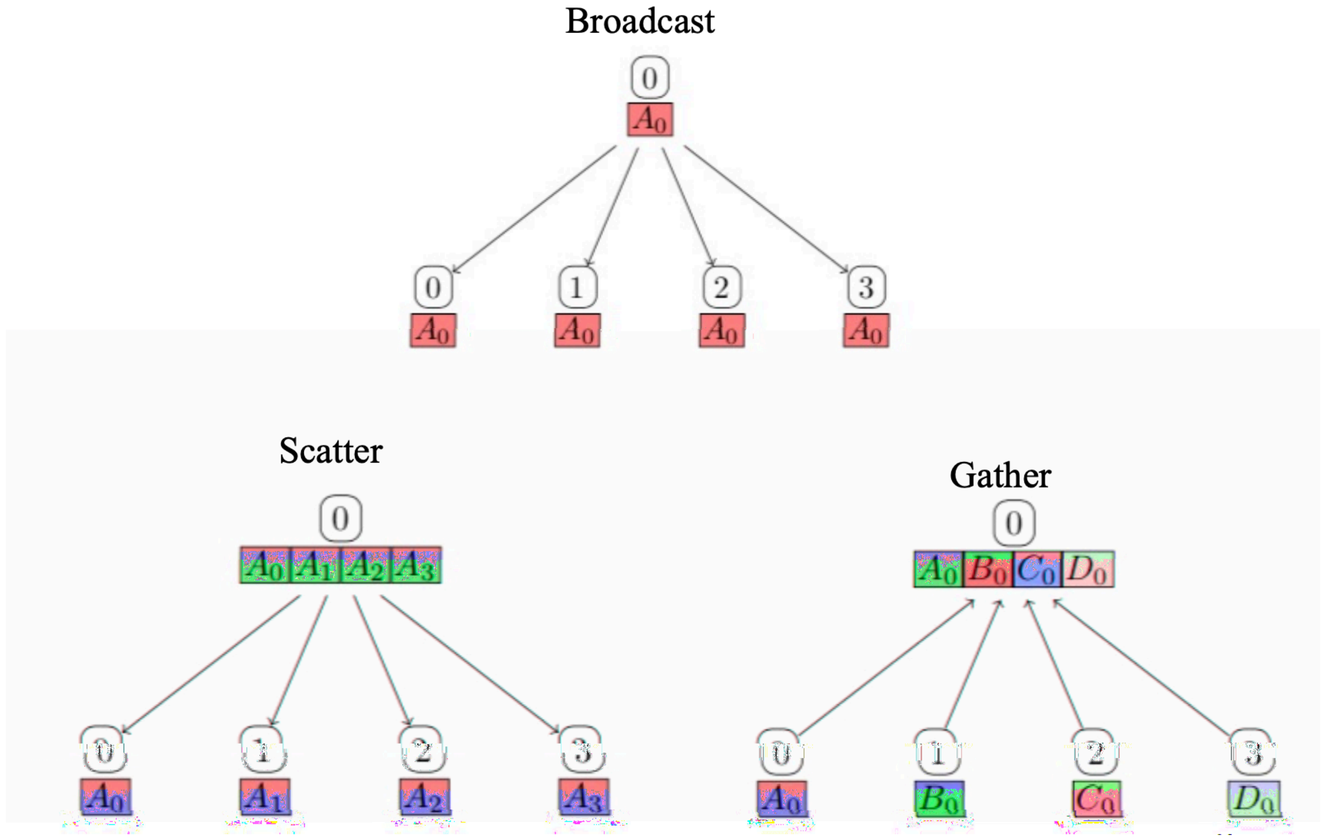
- Building blocks for algorithms that involve communication (sending data)
- Can sometimes include a small amount of computation (e.g., reduce, scan)



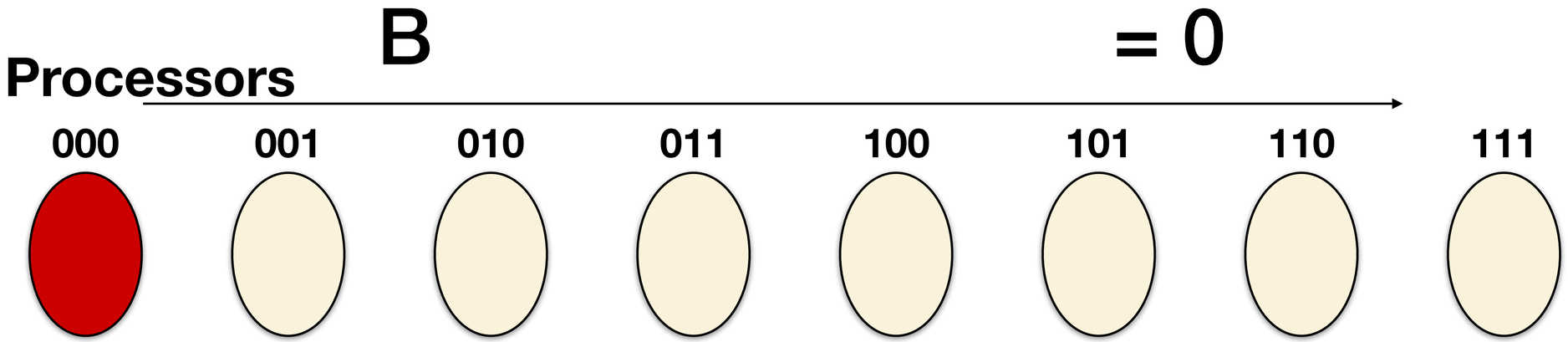
C C

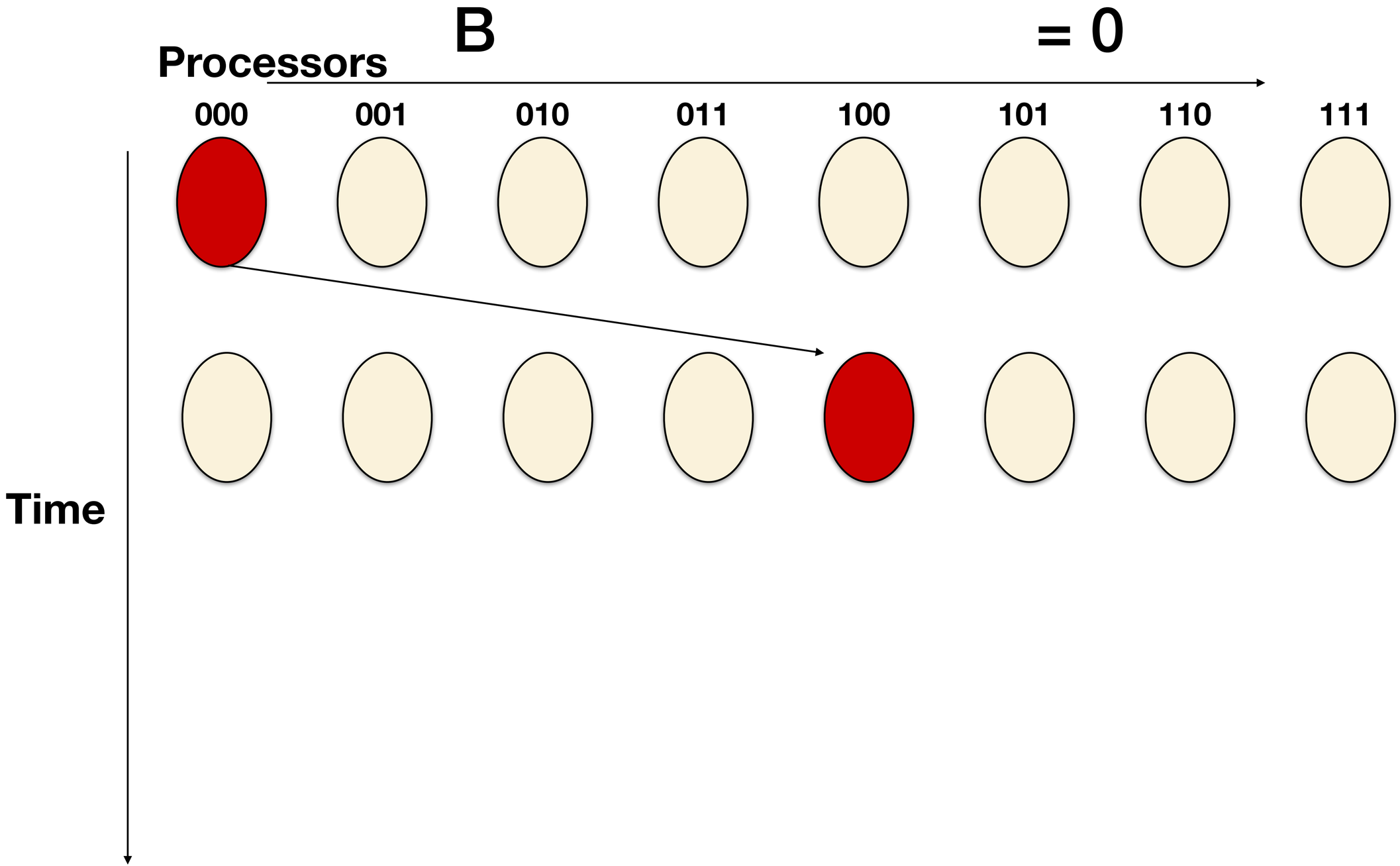
In collective communication, one process wants to communicate with multiple processes.

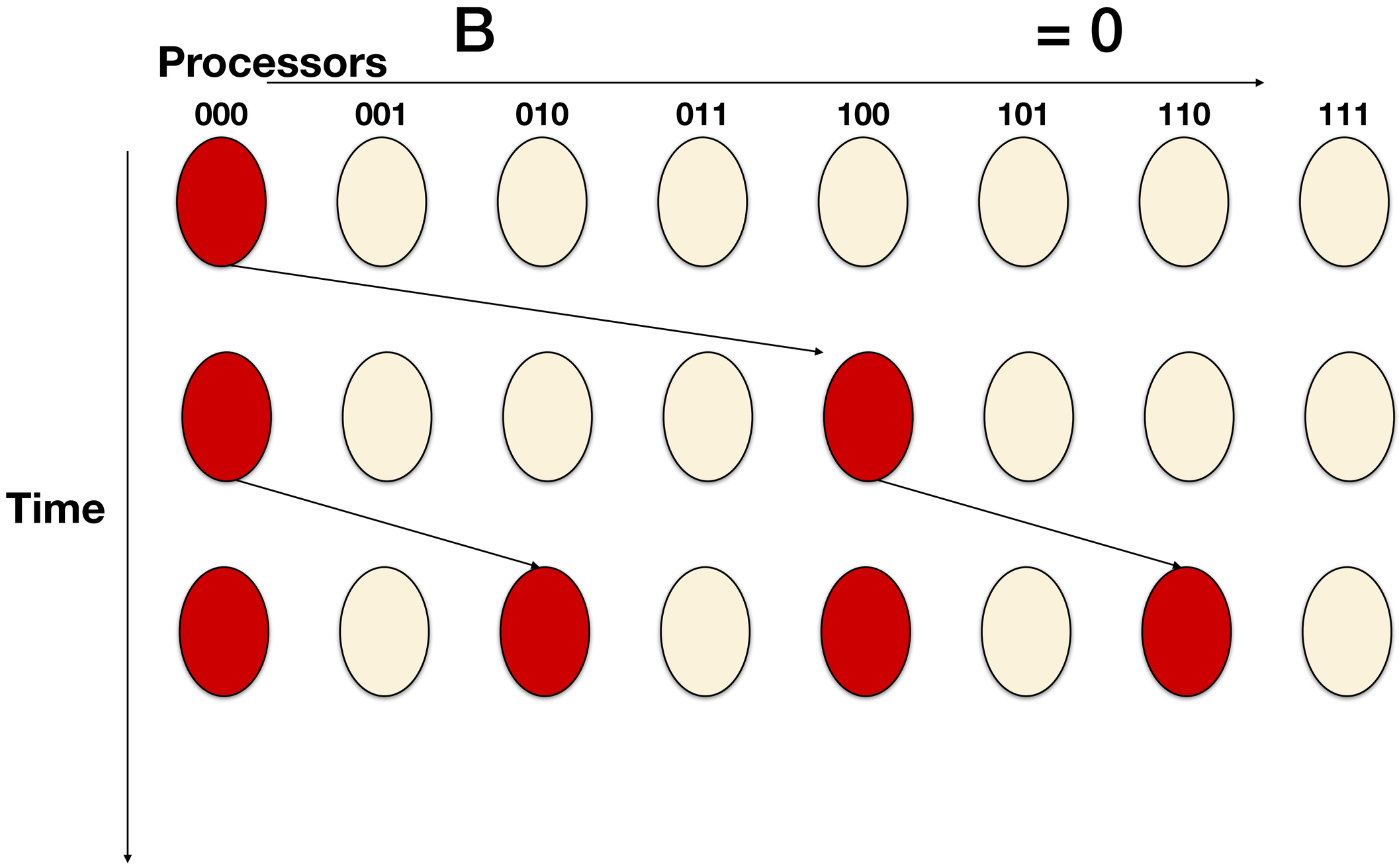
All collective operations must be called by all processes in the communicator



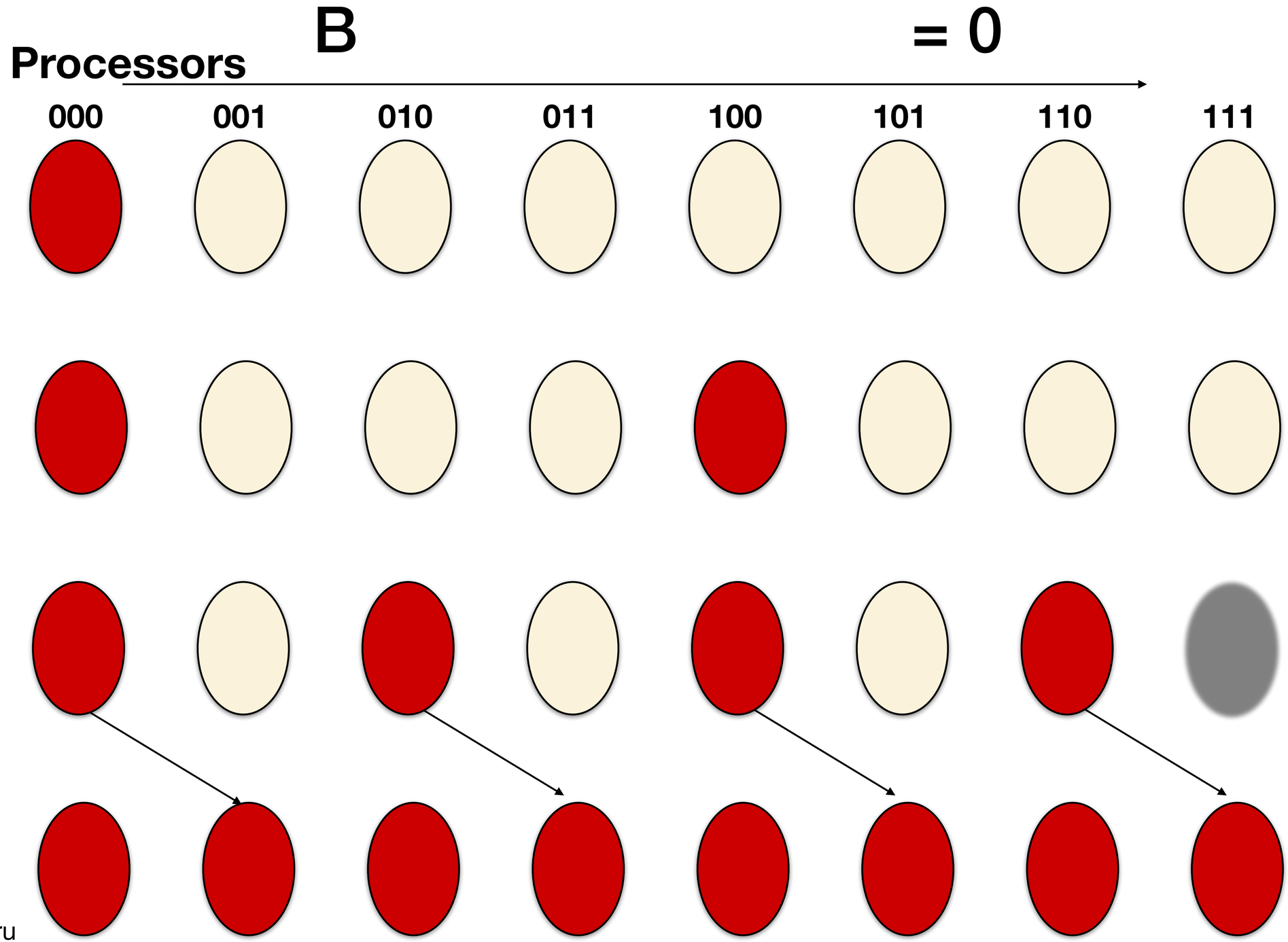
B ( )


B
= 0
Processors
000
001
010
011
100
101
110
111

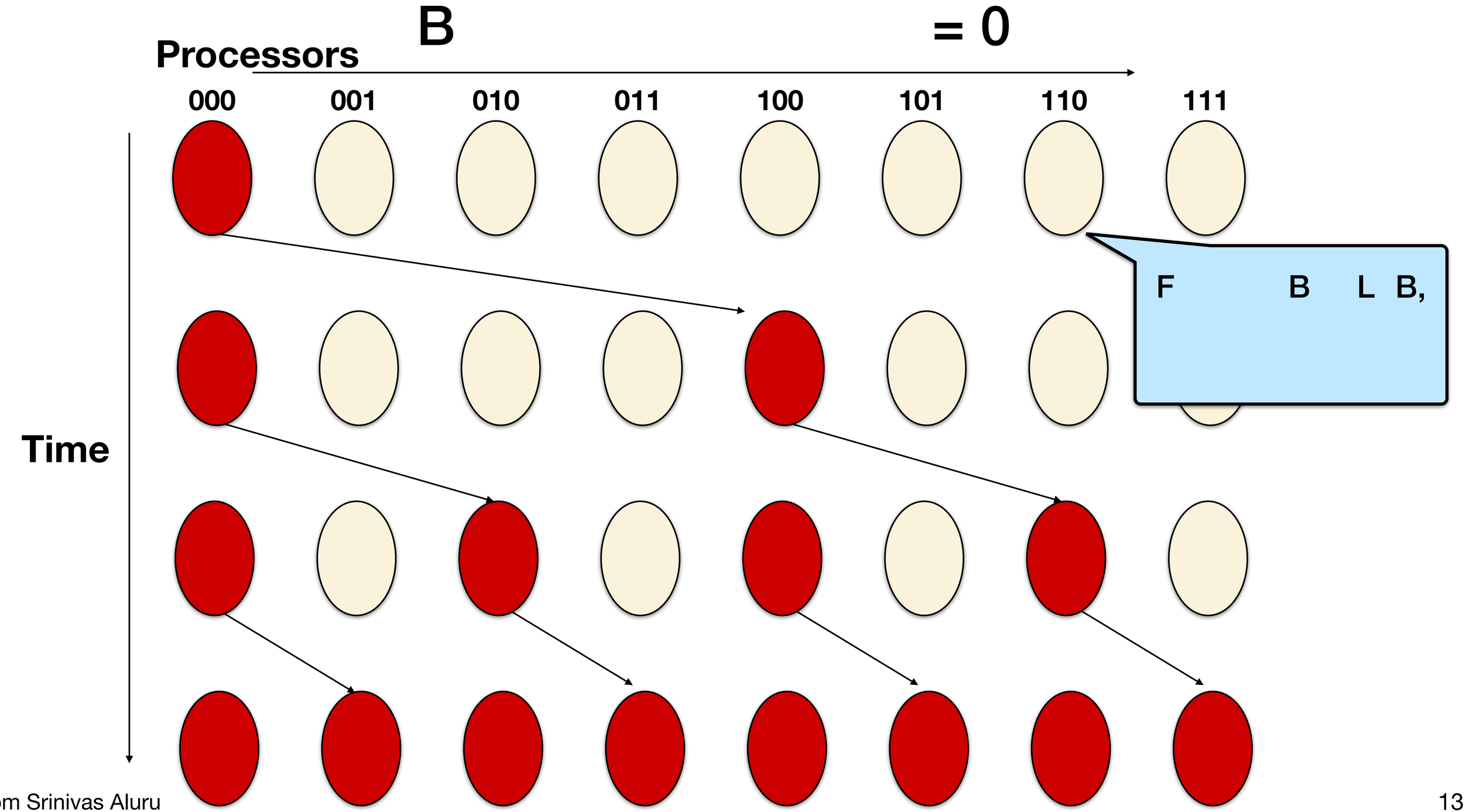

B
= 0
Processors
000
001
010
011
100
101
110
111
Time



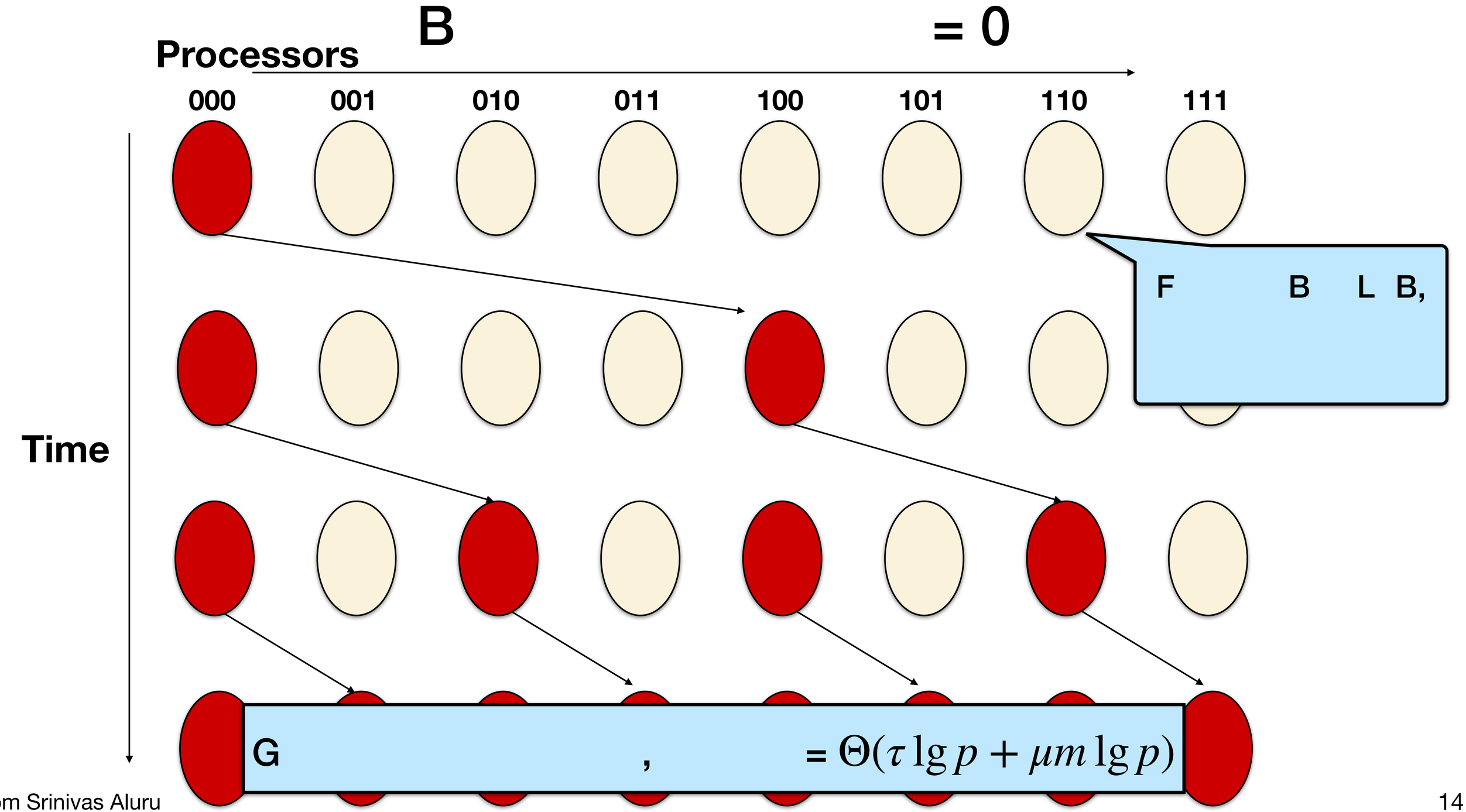
From Srinivas Aluru



From Srinivas Aluru



From Srinivas Aluru



From Srinivas Aluru

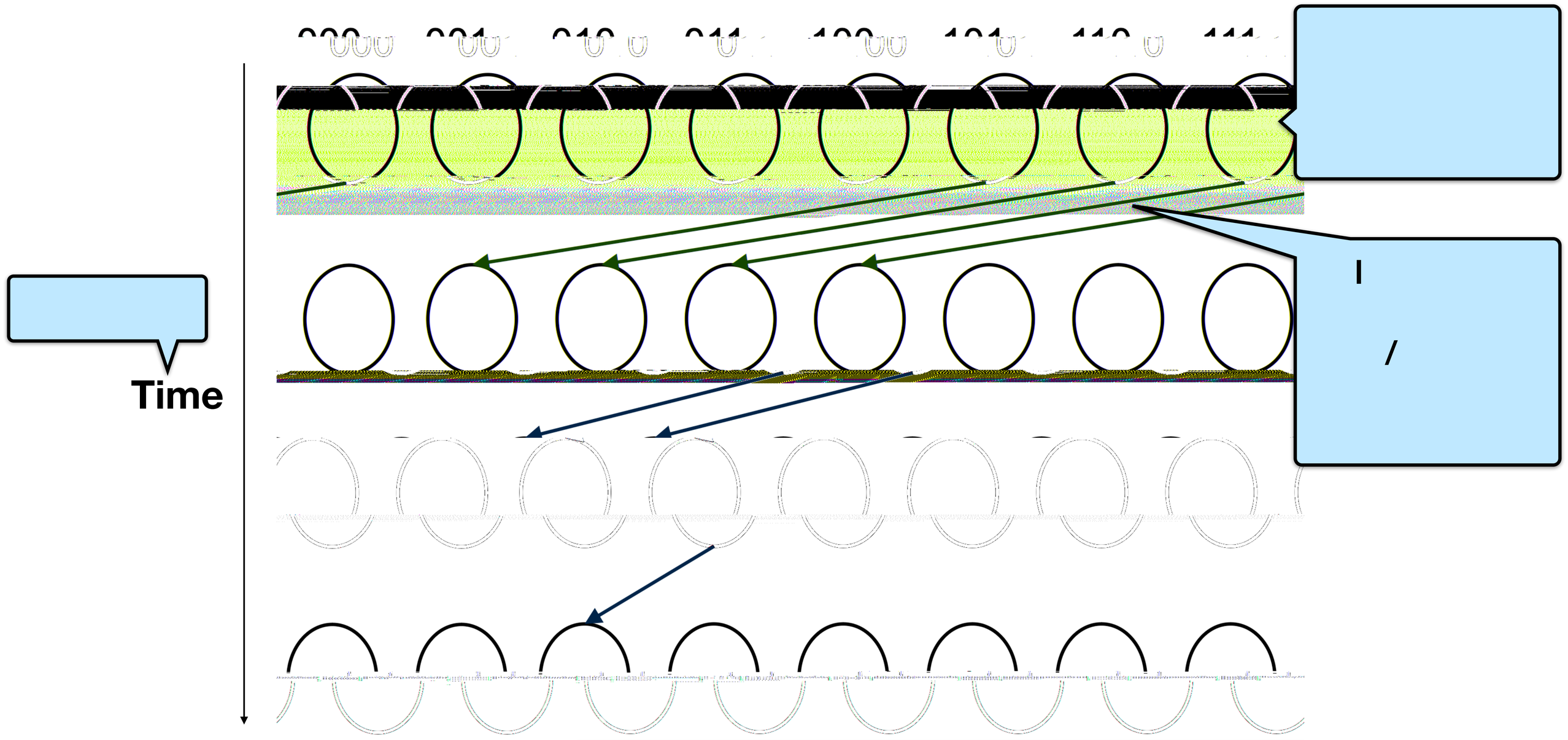
C

- Broadcast
- Reduce - aggregate all elements into one element using some operator
- Allreduce - all ranks get results of reduce
- Scan - prefix sum

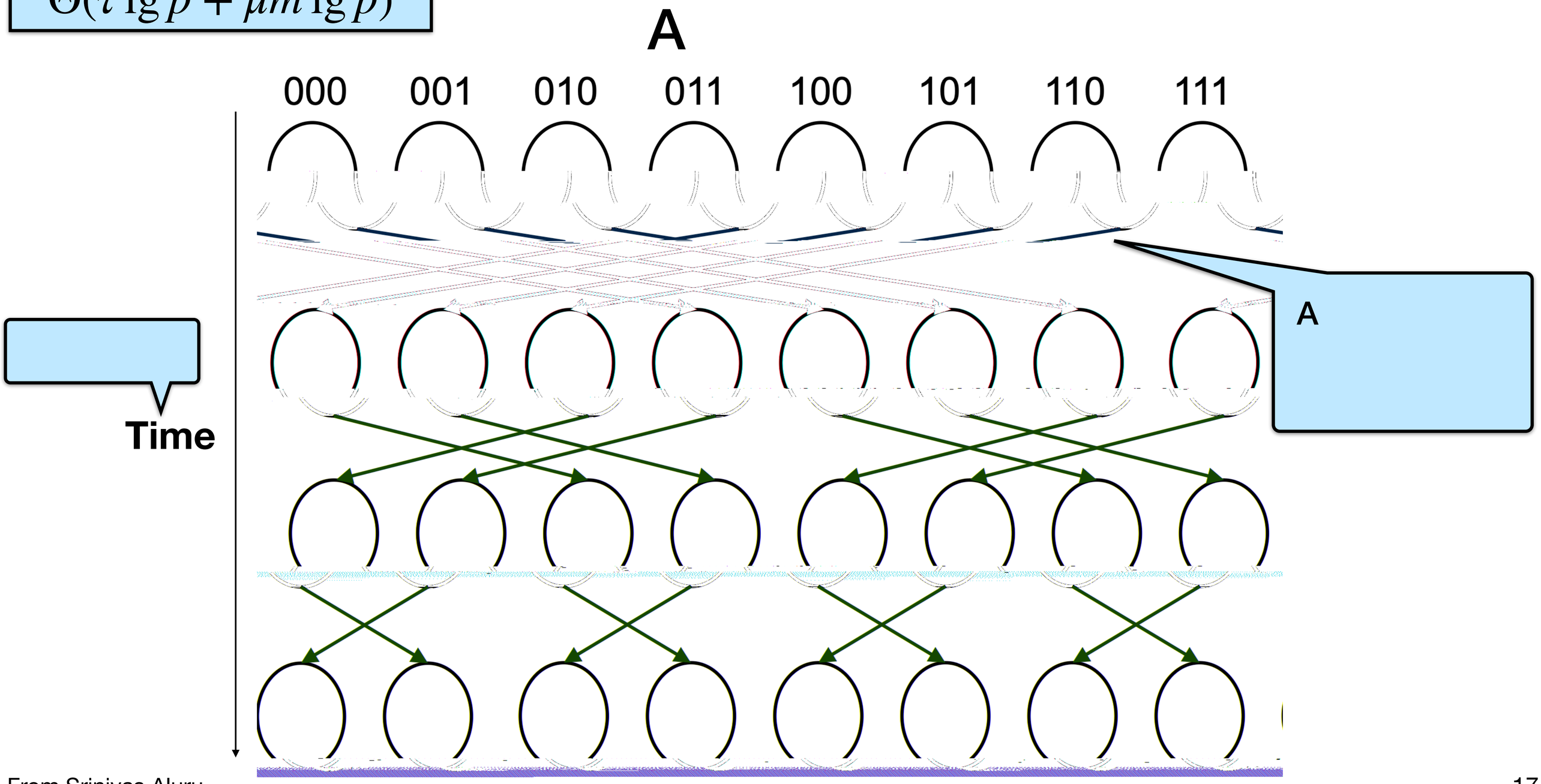
A

$$\Theta(\tau \lg p + \mu m \lg p)$$

$$\Theta(\tau \lg p + \mu m \lg p)$$



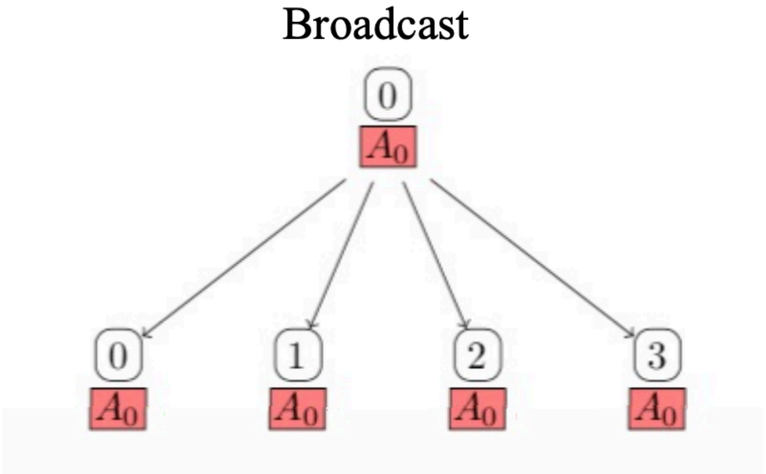
$$\Theta(\tau \lg p + \mu m \lg p)$$



I + E

MPI_Bcast

```
int MPI_Bcast(void* buffer,
              int count,
              MPI_Datatype datatype,
              int emitter_rank,
              MPI_Comm communicator);
```



- `MPI_Bcast` broadcasts a message from a process to all other processes in the same communicator.

- This is a collective operation; it must be called by **all processes in the communicator**.

https://rookiehpc.org/mpi/docs/mpi_bcast/index.html

MPI_Bcast Example

```
// Get my rank in the communicator
int world_rank;
MPI_Comm_rank(MPI_COMM_WORLD, &world_rank);

int data;
if (world_rank == 0) {
  data = 100;
  printf("Process 0 broadcasting data %d\n", data);
  MPI_Bcast(&data, 1, MPI_INT, 0, MPI_COMM_WORLD);
} else {
  MPI_Bcast(&data, 1, MPI_INT, 0, MPI_COMM_WORLD);
  printf("Process %d received data %d from root process\n", world_rank, data);
}
```

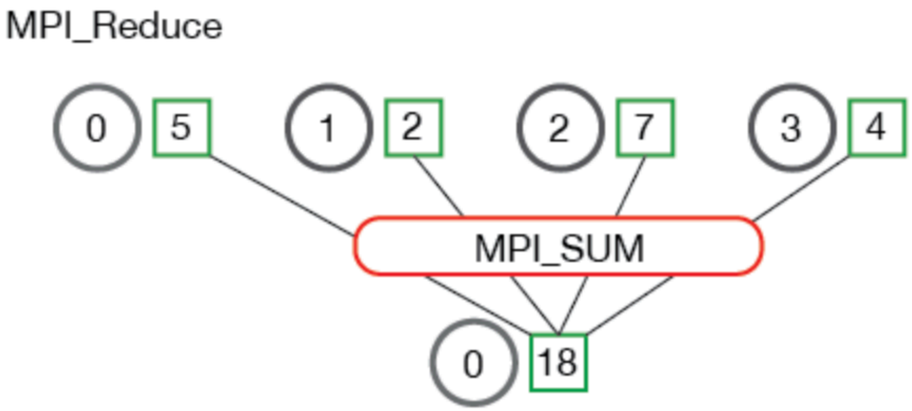
```
Process 0 broadcasting data 100
Process 2 received data 100 from root process
Process 3 received data 100 from root process
Process 1 received data 100 from root process
```

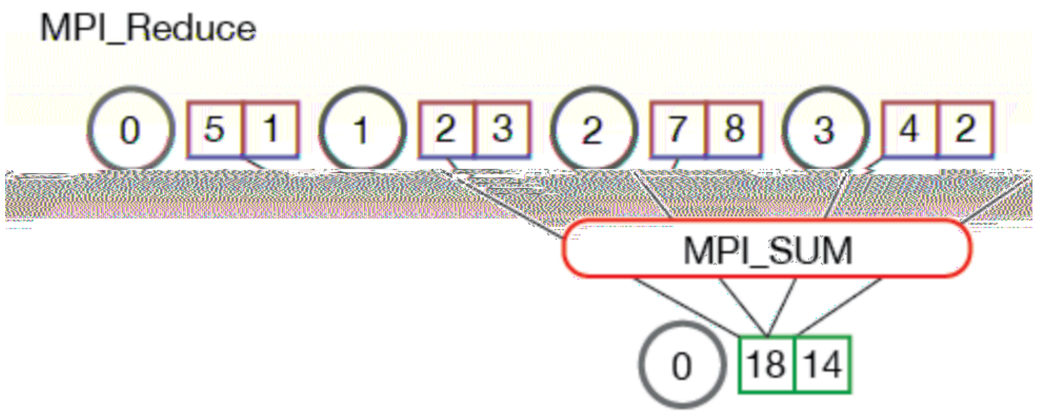
https://rookiehpc.org/mpi/docs/mpi_bcast/index.html

MPI_Reduce

MPI_Reduce reduces multiple numbers per process into an array element for each index.

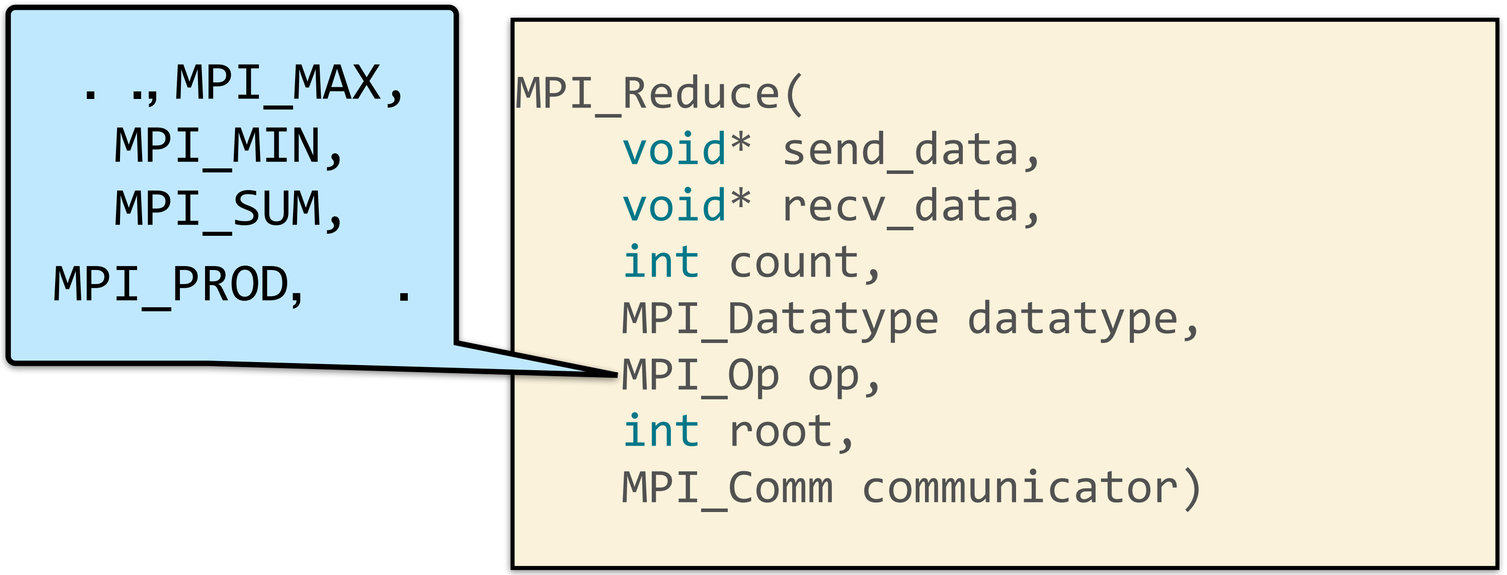
MPI_MAX, MPI_MIN, MPI_SUM, MPI_PROD, .





https://mpitutorial.com/tutorials/mpi-reduce-and-allreduce/

MPI_Reduce



- The `send_data` parameter is an array of elements that each process wants to reduce. Each `send_data` array has `count` elements.

- The `recv_data` is only relevant on the root and has # elements = `count`

A 4

A I

```
float *rand_nums = NULL;
rand_nums = create_rand_nums(num_elements_per_proc);

// Sum the numbers locally
float local_sum = 0;
for (int i = 0; i < num_elements_per_proc; i++) {
  local_sum += rand_nums[i];
}

// Print the random numbers on each process
printf("Local sum for process %d - %f, avg = %f\n",
       world_rank, local_sum, local_sum / num_elements_per_proc);

// Reduce all of the local sums into the global sum
float global_sum;
MPI_Reduce(&local_sum, &global_sum, 1, MPI_FLOAT, MPI_SUM, 0, MPI_COMM_WORLD);

// Print the result
if (world_rank == 0) {
  printf("Total sum = %f, avg = %f\n", global_sum,
         global_sum / (world_size * num_elements_per_proc));
}
```

https://mpitutorial.com/tutorials/mpi-reduce-and-allreduce/

A 4

A I

E

```
float *rand_nums = NULL;
rand_nums = create_rand_nums(num_elements_per_proc);

// Sum the numbers locally
float local_sum = 0;
for (int i = 0; i < num_elements_per_proc; i++) {
  local_sum += rand_nums[i];
}

// Print the random numbers on each process
printf("Local sum for process %d - %f, avg = %f\n",
       world_rank, local_sum, local_sum / num_elements_per_proc);

// Reduce all of the local sums into the global sum
float global_sum;
MPI_Reduce(&local_sum, &global_sum, 1, MPI_FLOAT, MPI_SUM, 0, MPI_COMM_WORLD);

// Print the result
if (world_rank == 0) {
  printf("Total sum = %f, avg = %f\n", global_sum,
         global_sum / (world_size * num_elements_per_proc));
}
```

A 4

A I

```
float *rand_nums = NULL;
rand_nums = create_rand_nums(num_elements_per_proc);

// Sum the numbers locally
float local_sum = 0;
for (int i = 0; i < num_elements_per_proc; i++) {
  local_sum += rand_nums[i];
}

// Print the random numbers on each process
printf("Local sum for process %d - %f, avg = %f\n",
       world_rank, local_sum, local_sum / num_elements_per_proc);

// Reduce all of the local sums into the global sum
float global_sum;
MPI_Reduce(&local_sum, &global_sum, 1, MPI_FLOAT, MPI_SUM, 0, MPI_COMM_WORLD);

// Print the result
if (world_rank == 0) {
  printf("Total sum = %f, avg = %f\n", global_sum,
         global_sum / (world_size * num_elements_per_proc));
}
```

E

L

A 4

A I

```
float *rand_nums = NULL;
rand_nums = create_rand_nums(num_elements_per_proc);

// Sum the numbers locally
float local_sum = 0;
for (int i = 0; i < num_elements_per_proc; i++) {
  local_sum += rand_nums[i];
}

// Print the random numbers on each process
printf("Local sum for process %d - %f, avg = %f\n",
       world_rank, local_sum, local_sum / num_elements_per_proc);

// Reduce all of the local sums into the global sum
float global_sum;
MPI_Reduce(&local_sum, &global_sum, 1, MPI_FLOAT, MPI_SUM, 0, MPI_COMM_WORLD);

// Print the result
if (world_rank == 0) {
  printf("Total sum = %f, avg = %f\n", global_sum,
         global_sum / (world_size * num_elements_per_proc));
}
```

E

L

A 4

A I

```
float *rand_nums = NULL;
rand_nums = create_rand_nums(num_elements_per_proc);

// Sum the numbers locally
float local_sum = 0;
for (int i = 0; i < num_elements_per_proc; i++) {
  local_sum += rand_nums[i];
}

// Print the random numbers on each process
printf("Local sum for process %d - %f, avg = %f\n",
       world_rank, local_sum, local_sum / num_elements_per_proc);

// Reduce all of the local sums into the global sum
float global_sum;
MPI_Reduce(&local_sum, &global_sum, 1, MPI_FLOAT, MPI_SUM, 0, MPI_COMM_WORLD);

// Print the result
if (world_rank == 0) {
  printf("Total sum = %f, avg = %f\n", global_sum,
         global_sum / (world_size * num_elements_per_proc));
}
```

E

L

G

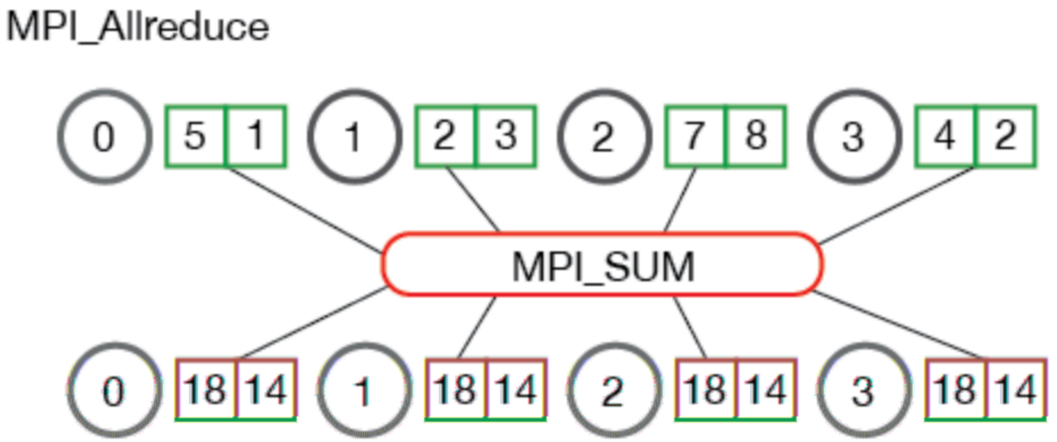


Output:

```
Local sum for process 0 - 51.385098, avg = 0.513851
Local sum for process 1 - 51.842468, avg = 0.518425
Local sum for process 2 - 49.684948, avg = 0.496849
Local sum for process 3 - 47.527420, avg = 0.475274
Total sum = 200.439941, avg = 0.501100
```

MPI_Allreduce

```
MPI_Allreduce(
    void* send_data,
    void* recv_data,
    int count,
    MPI_Datatype datatype,
    MPI_Op op,
    MPI_Comm communicator)
```

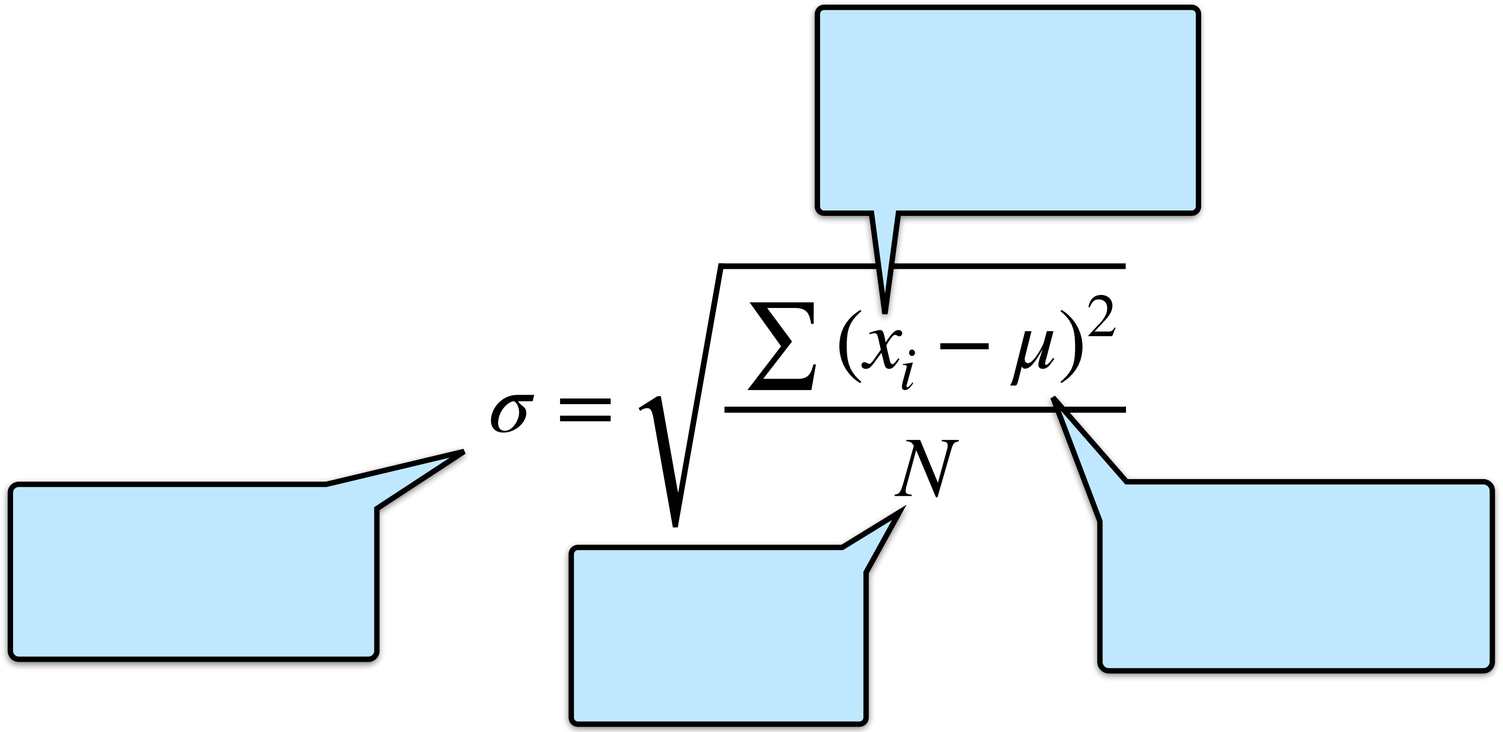


- `MPI_Allreduce` reduces the values and distributes the results to all processes.
- It is identical to `MPI_Reduce` except that it does not need a root.

https://mpitutorial.com/tutorials/mpi-reduce-and-allreduce/

E : D

Standard deviation is a measure of the dispersion of numbers from their mean.



How would you do it on a distributed set of numbers with MPI?

https://mpitutorial.com/tutorials/mpi-reduce-and-allreduce/

E : D MPI_Allreduce

```
rand_nums = create_rand_nums(num_elements_per_proc);

// Sum the numbers locally
float local_sum = 0;
for (int i = 0; i < num_elements_per_proc; i++) {
  local_sum += rand_nums[i];
}

// Reduce all of the local sums into the global sum in order to
// calculate the mean
float global_sum;
MPI_Allreduce(&local_sum, &global_sum, 1, MPI_FLOAT, MPI_SUM,
MPI_COMM_WORLD);
float mean = global_sum / (num_elements_per_proc * world_size);

// Compute the local sum of the squared differences from the mean
float local_sq_diff = 0;
for (i = 0; i < num_elements_per_proc; i++) {
  local_sq_diff += (rand_nums[i] - mean) * (rand_nums[i] - mean);
}
```

https://mpitutorial.com/tutorials/mpi-reduce-and-allreduce/

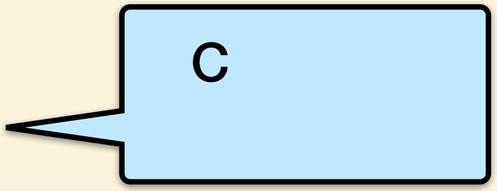
E : D MPI_Allreduce

```
rand_nums = create_rand_nums(num_elements_per_proc);

// Sum the numbers locally
float local_sum = 0;
for (int i = 0; i < num_elements_per_proc; i++) {
  local_sum += rand_nums[i];
}

// Reduce all of the local sums into the global sum in order to
// calculate the mean
float global_sum;
MPI_Allreduce(&local_sum, &global_sum, 1, MPI_FLOAT, MPI_SUM,
MPI_COMM_WORLD);
float mean = global_sum / (num_elements_per_proc * world_size);

// Compute the local sum of the squared differences from the mean
float local_sq_diff = 0;
for (i = 0; i < num_elements_per_proc; i++) {
  local_sq_diff += (rand_nums[i] - mean) * (rand_nums[i] - mean);
}
```



https://mpitutorial.com/tutorials/mpi-reduce-and-allreduce/

E : D MPI_Allreduce

```
rand_nums = create_rand_nums(num_elements_per_proc);

// Sum the numbers locally
float local_sum = 0;
for (int i = 0; i < num_elements_per_proc; i++) {
  local_sum += rand_nums[i];
}

// Reduce all of the local sums into the global sum in order to
// calculate the mean
float global_sum;
MPI_Allreduce(&local_sum, &global_sum, 1, MPI_FLOAT, MPI_SUM,
MPI_COMM_WORLD);
float mean = global_sum / (num_elements_per_proc * world_size);

// Compute the local sum of the squared differences from the mean
float local_sq_diff = 0;
for (i = 0; i < num_elements_per_proc; i++) {
  local_sq_diff += (rand_nums[i] - mean) * (rand_nums[i] - mean);
}
```

C

C

https://mpitutorial.com/tutorials/mpi-reduce-and-allreduce/

E : D MPI_Allreduce

```
rand_nums = create_rand_nums(num_elements_per_proc);

// Sum the numbers locally
float local_sum = 0;
for (int i = 0; i < num_elements_per_proc; i++) {
  local_sum += rand_nums[i];
}

// Reduce all of the local sums into the global sum in order to
// calculate the mean
float global_sum;
MPI_Allreduce(&local_sum, &global_sum, 1, MPI_FLOAT, MPI_SUM,
MPI_COMM_WORLD);
float mean = global_sum / (num_elements_per_proc * world_size);

// Compute the local sum of the squared differences from the mean
float local_sq_diff = 0;
for (i = 0; i < num_elements_per_proc; i++) {
  local_sq_diff += (rand_nums[i] - mean) * (rand_nums[i] - mean);
}
```

C

C

https://mpitutorial.com/tutorials/mpi-reduce-and-allreduce/

E : D MPI_Allreduce

```
// Reduce the global sum of the squared differences to the
// process and print off the answer
float global_sq_diff;
MPI_Reduce(&local_sq_diff, &global_sq_diff, 1, MPI_FLOAT, MPI_SUM, 0,
           MPI_COMM_WORLD);

// The standard deviation is the square root of the mean of the
// squared differences.
if (world_rank == 0) {
  float stddev = sqrt(global_sq_diff /
                      (num_elements_per_proc * world_size));
  printf("Mean - %f, Standard deviation = %f\n", mean, stddev);
}
```

https://mpitutorial.com/tutorials/mpi-reduce-and-allreduce/

E : D MPI_Allreduce

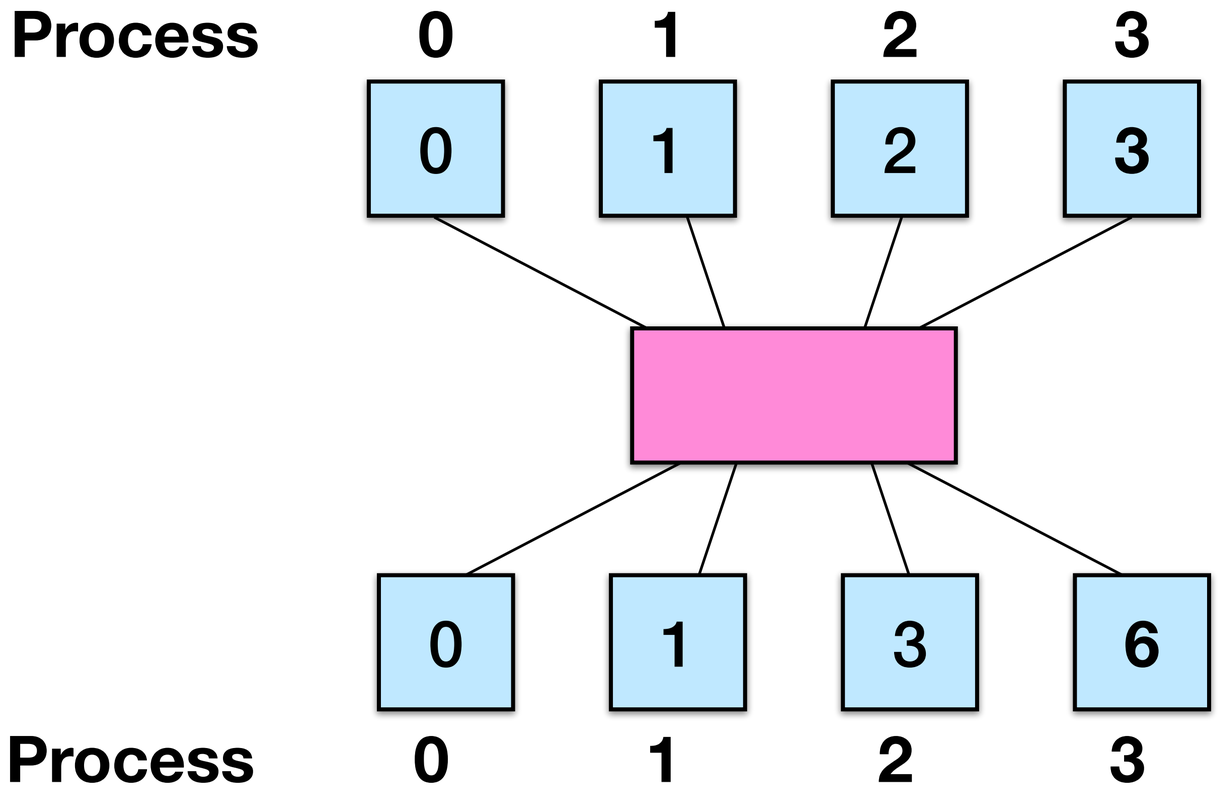
```
// Reduce the global sum of the squared differences to the
// process and print off the answer
float global_sq_diff;
MPI_Reduce(&local_sq_diff, &global_sq_diff, 1, MPI_FLOAT, MPI_SUM, 0,
           MPI_COMM_WORLD);

// The standard deviation is the square root of the mean of the
// squared differences.
if (world_rank == 0) {
  float stddev = sqrt(global_sq_diff /
                      (num_elements_per_proc * world_size));
  printf("Mean - %f, Standard deviation = %f\n", mean, stddev);
}
```

F

MPI_Scan

`MPI_Scan` performs an **inclusive scan** across all MPI processes in the given communicator.

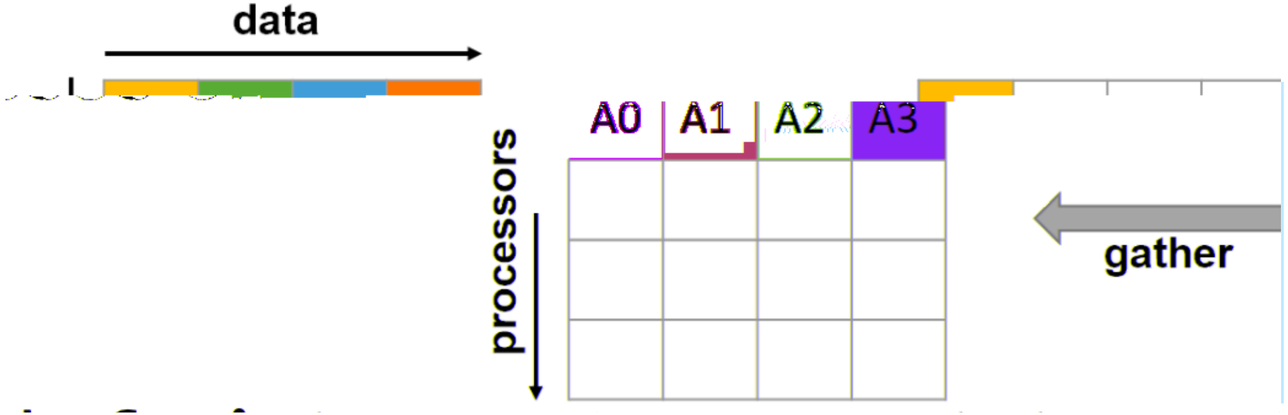


https://rookiehpc.org/mpi/docs/mpi_scan/index.html

/G /A A

G

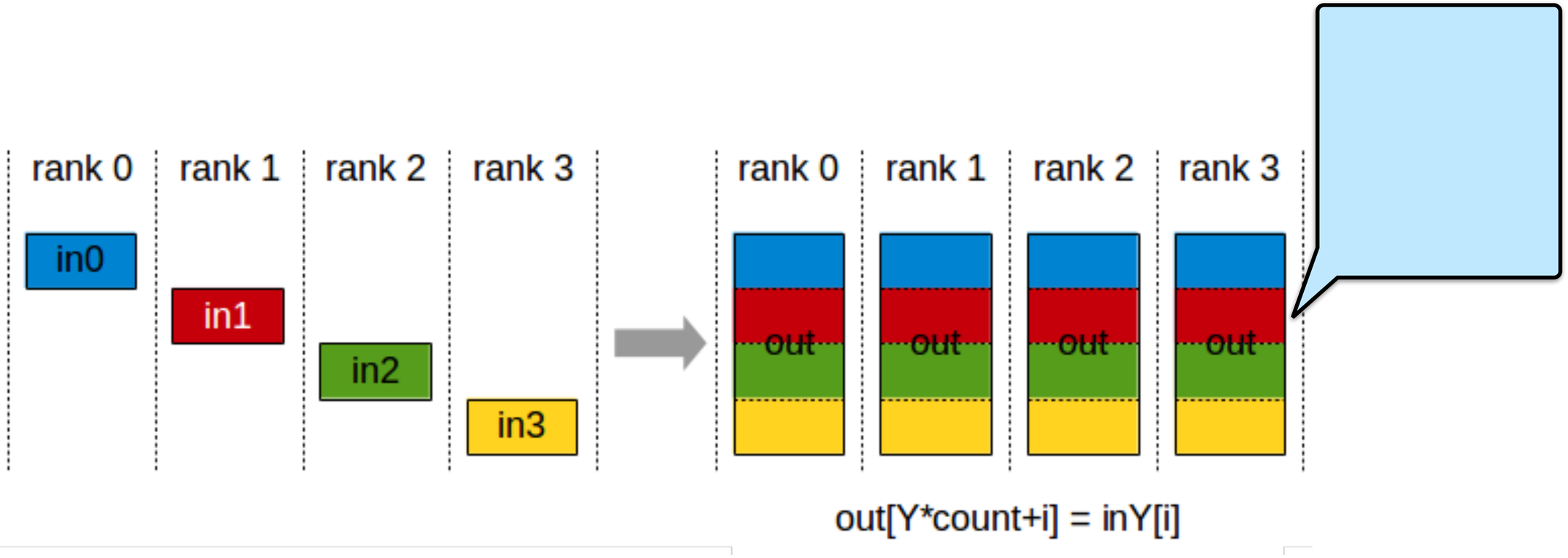
- Like reduce, but not aggregating them with an operator.
- One processor ends up with all the data that all of the processors are sending.
- Data appears **ordered by source processor** on the root processor.
- Data to be gathered should be much less than the problem size so that the whole algorithm is not worse than serial processing (when accounting for communication).



From Srinivas Aluru

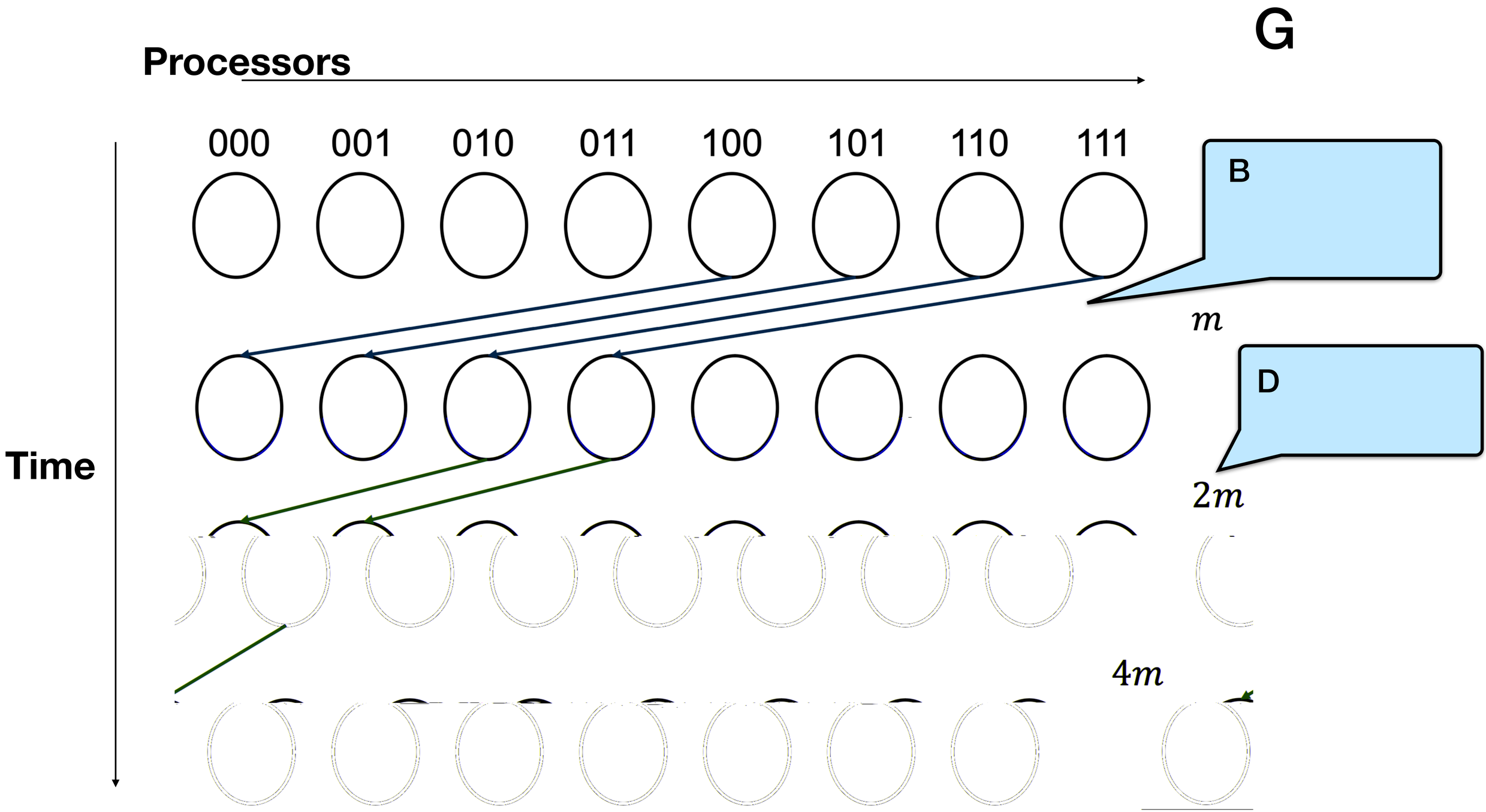
A G

Like gather, but every processor gets the entire answer.

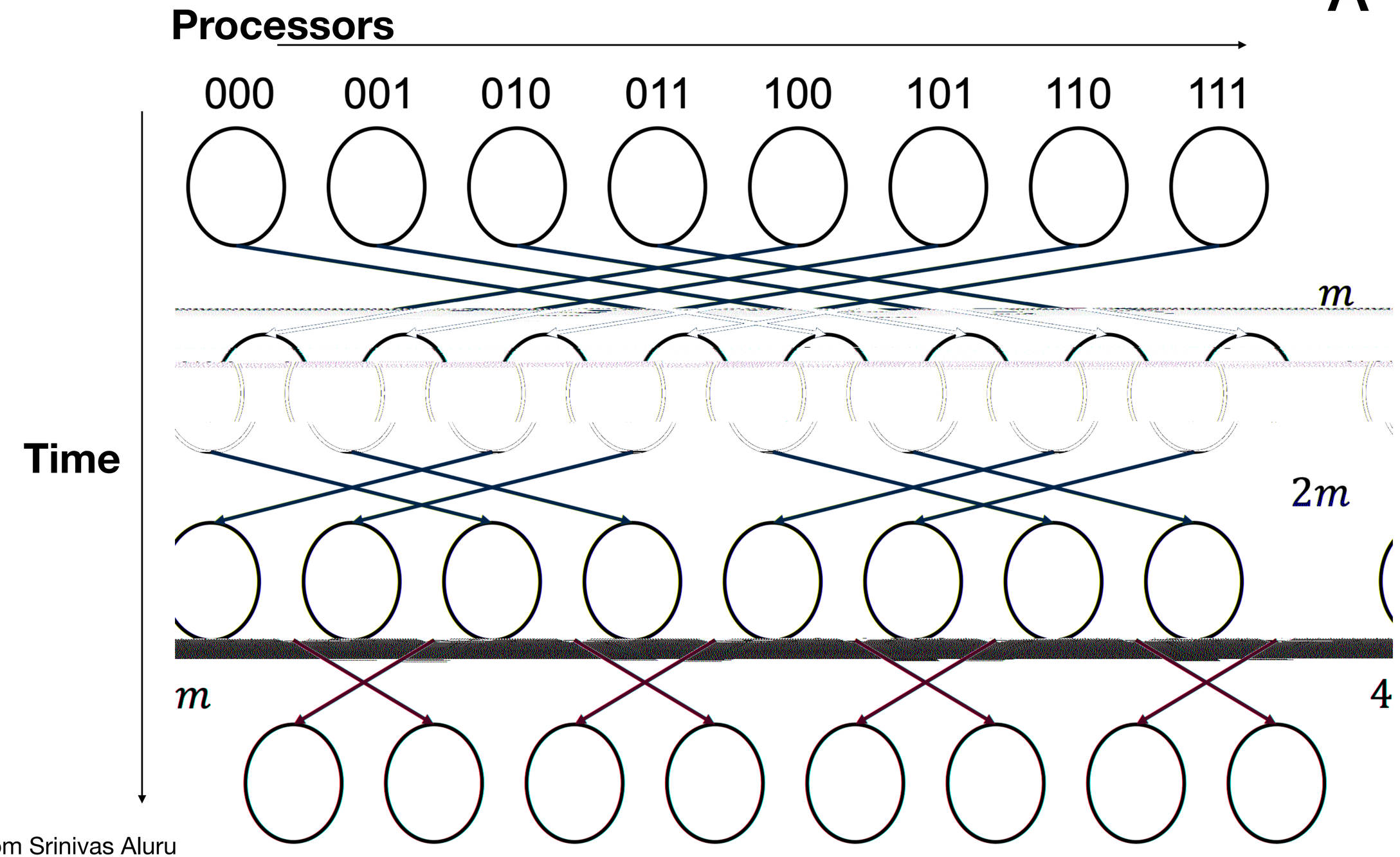


https://docs.nvidia.com/deeplearning/nccl/user-guide/docs/usage/collectives.html

From Srinivas Aluru


G
Processors
000
001
010
011
100
101
110
111
B
m
D
2m
4m
Time

A G



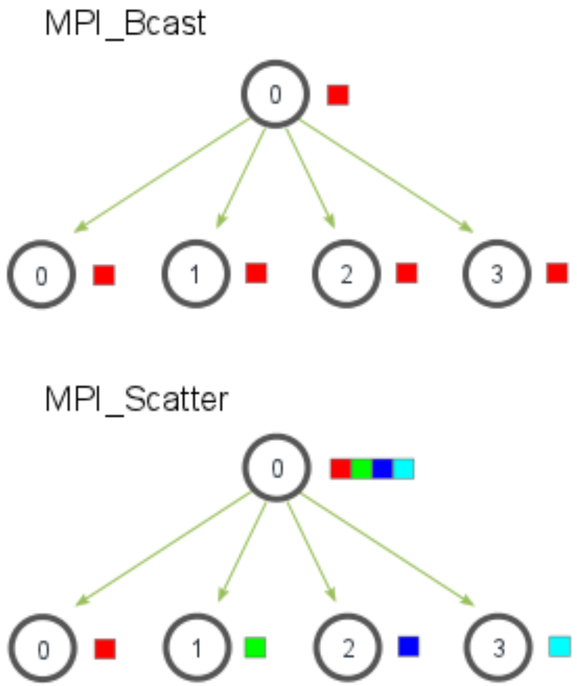
From Srinivas Aluru

A G /G

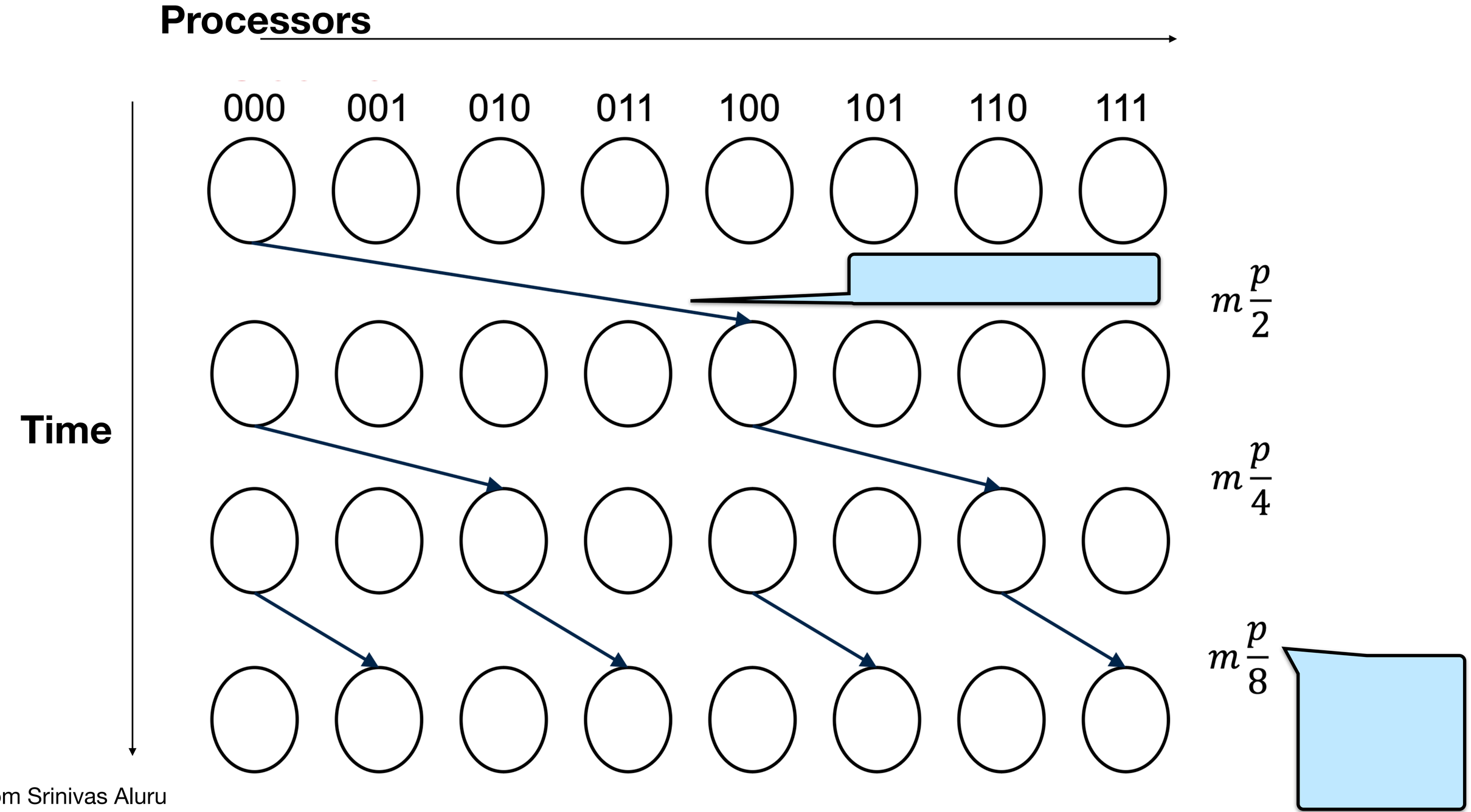
- Run-time for message size m
- Run-time $= \sum_{i=0}^{\log p - 1} (\tau + \mu \cdot m \cdot 2^i)$
- Run-time $= \Theta\left(\tau \log p + \mu \cdot m \cdot \left(1 + 2 + \cdots + \frac{p}{2}\right)\right)$
- Run-time $= \Theta(\tau \log p + \mu \cdot m \cdot p)$

- Scatter is the reverse of gather - **distribute data from root** and partition it across processors.

- Message size should be much less than input size for overall parallel speedup.



From Srinivas Aluru https://mpitutorial.com/tutorials/mpi-scatter-gather-and-allgather/

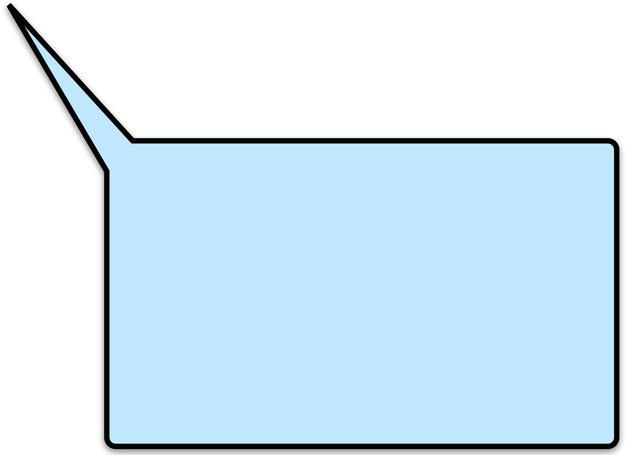

Processors
000
001
010
011
100
101
110
111
Time
$m\frac{p}{2}$
$m\frac{p}{4}$
$m\frac{p}{8}$

- Broadcast
- Reduce
- AllReduce
- Scan

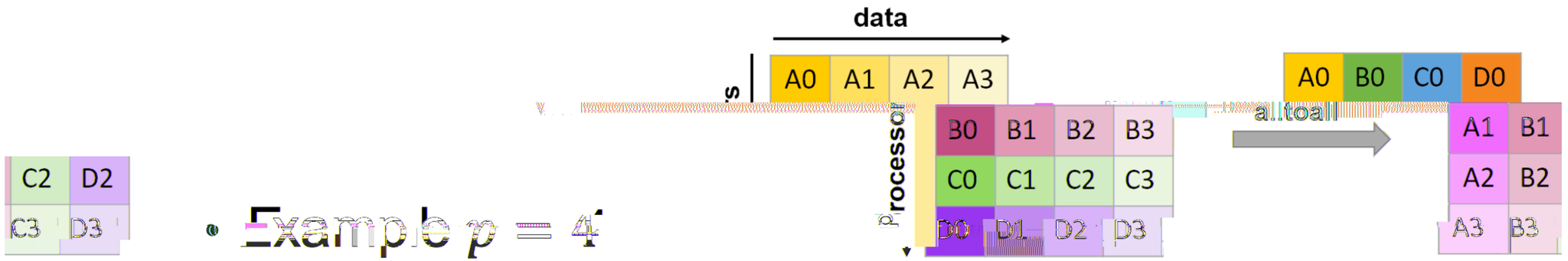
$\Theta(\tau \log p + \mu m \log p)$

- Gather
- AllGather
- Scatter

$\Theta(\tau \log p + \mu m p)$



A - -A



- Example $p = 4$

$$\begin{bmatrix} m_{30} \\ m_{31} \\ m_{32} \\ m_{33} \end{bmatrix} \quad \begin{matrix} 0: \\ 1: \\ 2: \\ 3: \end{matrix} \quad \begin{bmatrix} m_{00} & m_{01} & m_{02} & m_{03} \\ m_{10} & m_{11} & m_{12} & m_{13} \\ m_{20} & m_{21} & m_{22} & m_{23} \\ m_{30} & m_{31} & m_{32} & m_{33} \end{bmatrix} \Longrightarrow \begin{matrix} 0: \\ 1: \\ 2: \\ 3: \end{matrix} \begin{bmatrix} m_{00} & m_{10} & m_{20} & m_{30} \\ m_{01} & m_{11} & m_{21} & m_{31} \\ m_{02} & m_{12} & m_{22} & m_{32} \\ m_{03} & m_{13} & m_{23} & m_{33} \end{bmatrix}$$

: Message from P_i to P_j

- m_{ij}

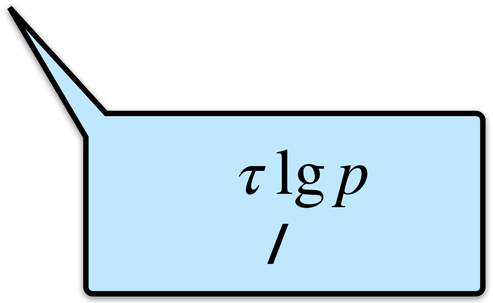
A A : A

Algorithm (for P_i)

```
for j=1 to (p-1) do
```

P_i sends $m_{i,(i+j) \bmod p}$ to $P_{(i+j) \bmod p}$

- $O(\tau \cdot p + \mu \cdot m \cdot p)$



- Broadcast
- Reduce
- AllReduce
- Scan

$\Theta(\tau \log p + \mu\, m \log p)$

- Gather
- AllGather
- Scatter

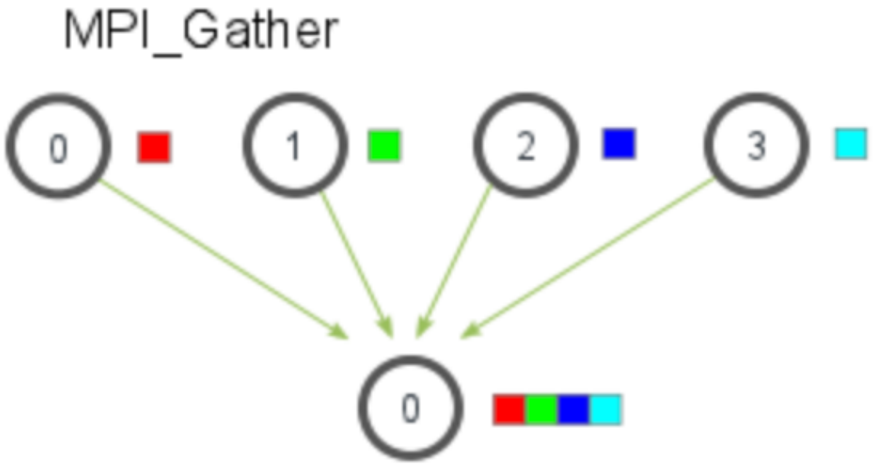
$\Theta(\tau \log p + \mu\, m\, p)$

- All_to_All
  - Arbitrary Permutations: $O(\tau \cdot p + \mu \cdot m \cdot p)$

I + E

MPI_Gather

```
MPI_Gather(
   void* send_data,
   int send_count,
   MPI_Datatype send_datatype,
   void* recv_data,
   int recv_count,
   MPI_Datatype recv_datatype,
   int root,
   MPI_Comm communicator)
```

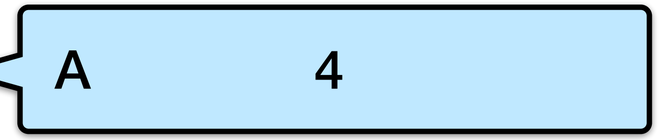


`MPI_Gather` is the **inverse of scatter** - it takes elements from many processes and gathers them into one process.

It is often used in parallel algorithms such as sorting and searching.

https://mpitutorial.com/tutorials/mpi-scatter-gather-and-allgather/

MPI_Gather Example




```
    // Get root and self rank
    int root_rank = 0;
    int my_rank;
    MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);

    // Define my value
    int my_value = my_rank * 100;
    printf("Process %d, my value = %d.\n", my_rank, my_value);

    if(my_rank == root_rank)
    {
        int buffer[4];
        MPI_Gather(&my_value, 1, MPI_INT, buffer, 1, MPI_INT, root_rank, MPI_COMM_WORLD);
        printf("Values collected on process %d: %d, %d, %d, %d.\n", my_rank, buffer[0],
buffer[1], buffer[2], buffer[3]);
    }
    else
    {
        MPI_Gather(&my_value, 1, MPI_INT, NULL, 0, MPI_INT, root_rank, MPI_COMM_WORLD);
    }
```

https://rookiehpc.org/mpi/docs/mpi_gather/index.html

MPI_Gather Example

```
    // Get root and self rank
    int root_rank = 0;
    int my_rank;
    MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);

    // Define my value
    int my_value = my_rank * 100;
    printf("Process %d, my value = %d.\n", my_rank, my_value);

    if(my_rank == root_rank)
    {
        int buffer[4];
        MPI_Gather(&my_value, 1, MPI_INT, buffer, 1, MPI_INT, root_rank, MPI_COMM_WORLD);
        printf("Values collected on process %d: %d, %d, %d, %d.\n", my_rank, buffer[0],
buffer[1], buffer[2], buffer[3]);
    }
    else
    {
        MPI_Gather(&my_value, 1, MPI_INT, NULL, 0, MPI_INT, root_rank, MPI_COMM_WORLD);
    }
```

https://rookiehpc.org/mpi/docs/mpi_gather/index.html

MPI_Gather Example

A 4

```
    // Get root and self rank
    int root_rank = 0;
    int my_rank;
    MPI_Comm_rank(MPI_COMM_WORLD, &my

    // Define my value
    int my_value = my_rank * 100;
    printf("Process %d, my valu = %d.\n", my_rank, my_value);

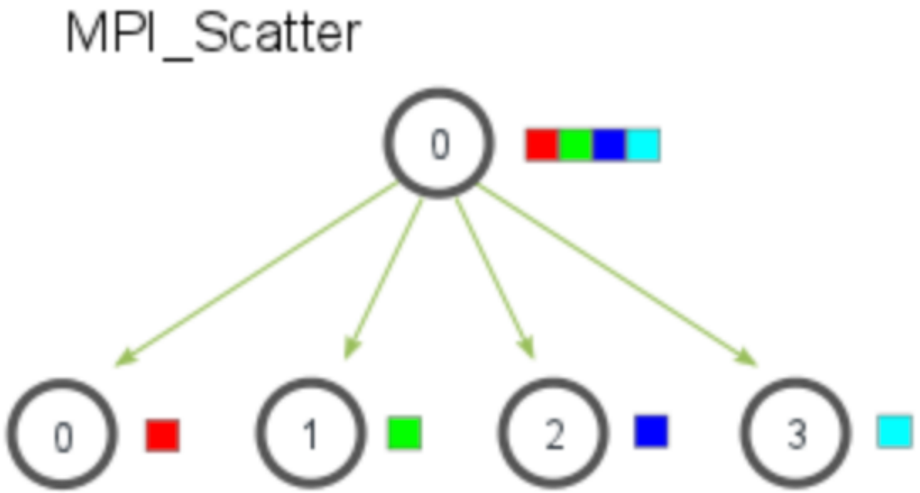
    if(my_rank == root_rank)
    {
        int buffer[4];
        MPI_Gather(&my_value, 1, MPI_INT, buffer, 1, MPI_INT, root_rank, MPI_COMM_WORLD);
        printf("Values collected on process %d: %d, %d, %d, %d.\n", my_rank, buffer[0],
buffer[1], buffer[2], buffer[3]);
    }
    else
    {
        MPI_Gather(&my_value, 1, MPI_INT, NULL, 0, MPI_INT, root_rank, MPI_COMM_WORLD);
    }
```

: 0: 0, 100, 100, 300

https://rookiehpc.org/mpi/docs/mpi_gather/index.html

MPI_Scatter

```
MPI_Scatter(
   void* send_data,
   int send_count,
   MPI_Datatype send_datatype,
   void* recv_data,
   int recv_count,
   MPI_Datatype recv_datatype,
   int root,
   MPI_Comm communicator)
```



`MPI_Scatter` takes an array of elements and **distributes the elements in order of process rank**. Process 0 gets the first chunk, process 1 gets the next, and so on.

https://mpitutorial.com/tutorials/mpi-scatter-gather-and-allgather/

MPI_Scatter Example

A 4

```
    // Determine root's rank
    int root_rank = 0;

    // Get my rank
    int my_rank;
    MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);

    // Define my value
    int my_value;

    if(my_rank == root_rank)
    {
        int buffer[4] = {0, 100, 200, 300};
        printf("Values to scatter from process %d: %d, %d, %d, %d.\n", my_rank, buffer[0],
buffer[1], buffer[2], buffer[3]);
        MPI_Scatter(buffer, 1, MPI_INT, &my_value, 1, MPI_INT, root_rank, MPI_COMM_WORLD);
    }
    else
    {
        MPI_Scatter(NULL, 1, MPI_INT, &my_value, 1, MPI_INT, root_rank, MPI_COMM_WORLD);
    }

    printf("Process %d received value = %d.\n", my_rank, my_value);
```

https://rookiehpc.org/mpi/docs/mpi_scatter/index.html

MPI_Scatter Example

A 4

```
    // Determine root's rank
    int root_rank = 0;

    // Get my rank
    int my_rank;
    MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);

    // Define my value
    int my_value;

    if(my_rank == root_rank)
    {
        int buffer[4] = {0, 100, 200, 300};
        printf("Values to scatter from process %d: %d, %d, %d, %d.\n", my_rank, buffer[0],
buffer[1], buffer[2], buffer[3]);
        MPI_Scatter(buffer, 1, MPI_INT, &my_value, 1, MPI_INT, root_rank, MPI_COMM_WORLD);
    }
    else
    {
        MPI_Scatter(NULL, 1, MPI_INT, &my_value, 1, MPI_INT, root_rank, MPI_COMM_WORLD);
    }

    printf("Process %d received value = %d.\n", my_rank, my_value);
```

0: 0, 100, 200, 300

https://rookiehpc.org/mpi/docs/mpi_scatter/index.html

MPI_Scatter Example

A 4

```
    // Determine root's rank
    int root_rank = 0;

    // Get my rank
    int my_rank;
    MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);

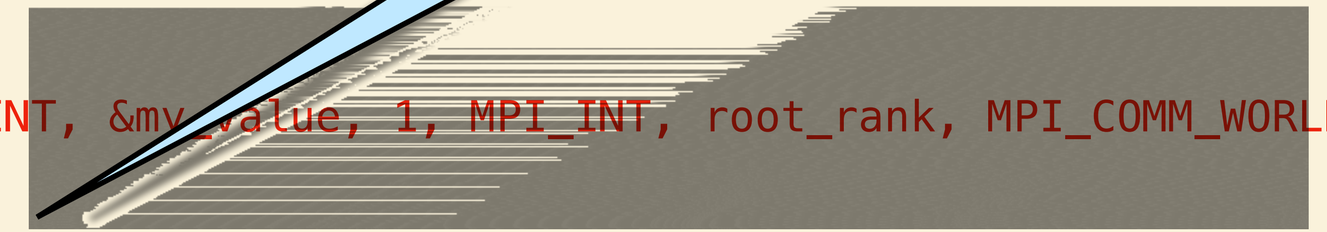
    // Define my value
    int my_value;

    if(my_rank == root_rank)
    {
        int buffer[4] = {0, 100, 200, 300};
        printf("Values to scatter from process %d: 
er[0],
buffer[1], buffer[2], buffer[3]);
        MPI_Scatter(buffer, 1, MPI_INT, &my_value,
ORLD);
    }
    else
    {
        MPI_Scatter(NULL, 1, MPI_INT, &my_value, 1, MPI_INT, root_rank, MPI_COMM_WORLD);
    }

    printf("Process %d received value = %d.\n", my_rank, my_value);
```

0: 0, 100, 200, 300

0	= 0
1	= 100
2	= 200
3	= 300



https://rookiehpc.org/mpi/docs/mpi_scatter/index.html

E : A G /

```
if (world_rank == 0) {
  rand_nums = create_rand_nums(elements_per_proc * world_size);
}

// Create a buffer that will hold a subset of the random numbers
float *sub_rand_nums = malloc(sizeof(float) * elements_per_proc);

// Scatter the random numbers to all processes
MPI_Scatter(rand_nums, elements_per_proc, MPI_FLOAT, sub_rand_nums,
            elements_per_proc, MPI_FLOAT, 0, MPI_COMM_WORLD);

// Compute the average of your subset
float sub_avg = compute_avg(sub_rand_nums, elements_per_proc);
// Gather all partial averages down to the root process
float *sub_avgs = NULL;
if (world_rank == 0) {
  sub_avgs = malloc(sizeof(float) * world_size);
}
…
```

https://mpitutorial.com/tutorials/mpi-scatter-gather-and-allgather/

E : A G /

```
if (world_rank == 0) {
  rand_nums = create_rand_num
}

// Create a buffer that will hold a subset of the random numbers
float *sub_rand_nums = malloc(sizeof(float) * elements_per_proc);

// Scatter the random numbers to all processes
MPI_Scatter(rand_nums, elements_per_proc, MPI_FLOAT, sub_rand_nums,
            elements_per_proc, MPI_FLOAT, 0, MPI_COMM_WORLD);

// Compute the average of your subset
float sub_avg = compute_avg(sub_rand_nums, elements_per_proc);
// Gather all partial averages down to the root process
float *sub_avgs = NULL;
if (world_rank == 0) {
  sub_avgs = malloc(sizeof(float) * world_size);
}
…
```

E

https://mpitutorial.com/tutorials/mpi-scatter-gather-and-allgather/

E : A G /

```
if (world_rank == 0) {
  rand_nums = create_rand_nums(elements_per_proc * world_size);
}

// Create a buffer that will hold a subs
float *sub_rand_nums = malloc(sizeof(flo

// Scatter the random numbers to all processes
MPI_Scatter(rand_nums, elements_per_proc, MPI_FLOAT, sub_rand_nums,
            elements_per_proc, MPI_FLOAT, 0, MPI_COMM_WORLD);

// Compute the average of your subset
float sub_avg = compute_avg(sub_rand_nums, elements_per_proc);
// Gather all partial averages down to the root process
float *sub_avgs = NULL;
if (world_rank == 0) {
  sub_avgs = malloc(sizeof(float) * world_size);
}
…
```

A ,

https://mpitutorial.com/tutorials/mpi-scatter-gather-and-allgather/

E : A G /

```
if (world_rank == 0) {
  rand_nums = create_rand_nums(elements_per_proc * world_size);
}

// Create a buffer that will hold a subset of the random numbers
float *sub_rand_nums = malloc(sizeof(float) * elements_per_proc);

// Scatter the random numbers to all processes
MPI_Scatter(rand_nums, elements_per_proc, MPI_FLOAT, sub_rand_nums,
            elements_per_proc, MPI_FLOAT, 0, MPI_COMM_WORLD);

// Compute the average of your subset
float sub_avg = compute_avg(sub_rand_nums, elements_per_proc);
// Gather all partial averages down to the root process
float *sub_avgs = NULL;
if (world_rank == 0) {
  sub_avgs = malloc(sizeof(float) * world_size);
}
…
```

E

E : A G /

```
if (world_rank == 0) {
  rand_nums = create_rand_nums(elements_per_proc * world_size);
}

// Create a buffer that will hold a subset of the random numbers
float *sub_rand_nums = malloc(sizeof(float) * elements_per_proc);

// Scatter the random numbers to all processes
MPI_Scatter(rand_nums, elements_per_proc, MPI_FLOAT, sub_rand_nums,
            elements_per_proc, MPI_FLOAT, 0, MPI_COMM_WORLD);

// Compute the average of your subset
float sub_avg = compute_avg(sub_rand_nums, elements_per_proc);
// Gather all partial averages down to the root process
float *sub_avgs = NULL;
if (world_rank == 0) {
  sub_avgs = malloc(sizeof(float) * world_size);
}
…
```



E : A G /

```
…
if (world_rank == 0) {
  sub_avgs = malloc(sizeof(float) * world_size);
}
MPI_Gather(&sub_avg, 1, MPI_FLOAT, sub_avgs, 1, MPI_FLOAT, 0,
           MPI_COMM_WORLD);

// Compute the total average of all numbers.
if (world_rank == 0) {
  float avg = compute_avg(sub_avgs, world_size);
  printf(“Avg computed across original data is %f\n”, avg);
}
```



```
Avg computed across original data is 0.479736
```

E : A G /

```
…
if (world_rank == 0) {
  sub_avgs = malloc(sizeof(float) * world_size);
}
MPI_Gather(&sub_avg, 1, MPI_FLOAT, sub_avgs, 1, MPI_FLOAT, 0,
           MPI_COMM_WORLD);

// Compute the total average of all numbers.
if (world_rank == 0) {
  float avg = compute_avg(sub_avgs, world_size);
  printf(“Avg computed across original data is %f\n”, avg);
}
```



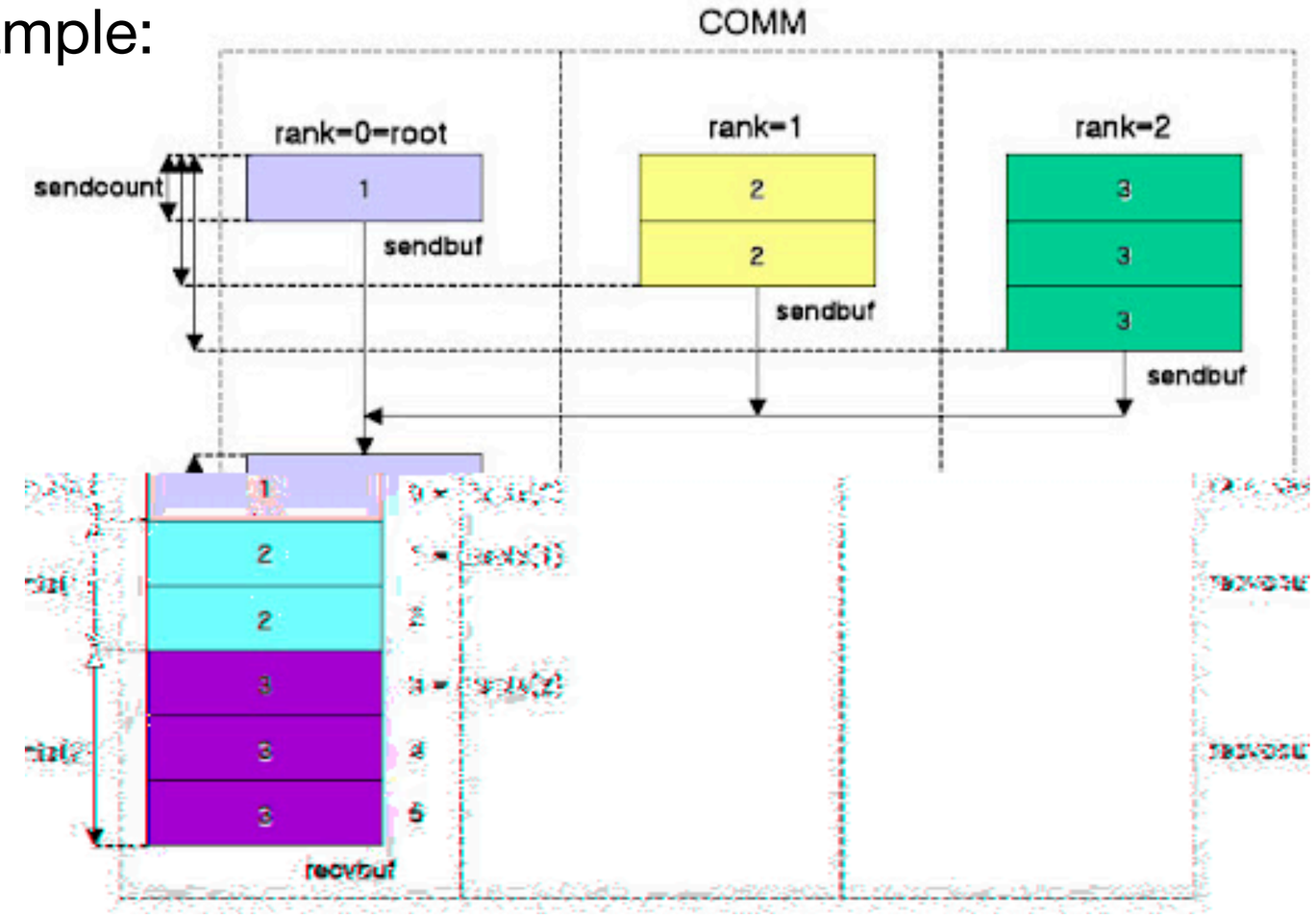


```
Avg computed across original data is 0.479736
```

L C

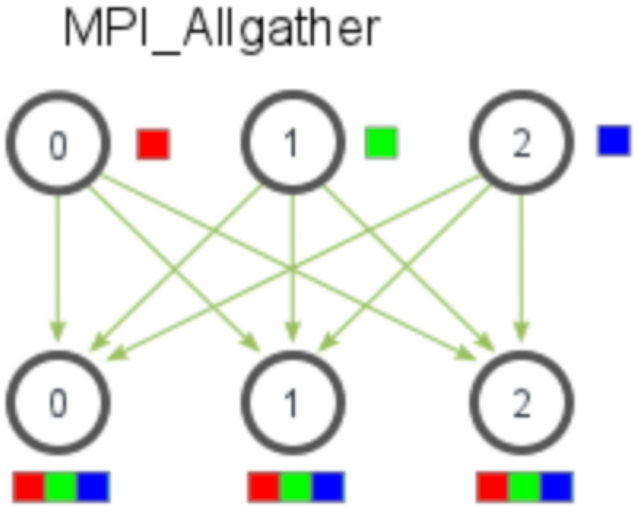
Variable length: e.g. `gatherv, scatterv,` etc.

`gatherv` example:



MPI_Allgather

```
MPI_Allgather(
   void* send_data,
   int send_count,
   MPI_Datatype send_datatype,
   void* recv_data,
   int recv_count,
   MPI_Datatype recv_datatype,
   MPI_Comm communicator)
```



- `MPI_Allgather` implements **many-to-many communication** - similar to `MPI_Gather` then `MPI_Bcast`.

- The function declaration is the same as `MPI_Gather` except that there is no root.

https://mpitutorial.com/tutorials/mpi-scatter-gather-and-allgather/

MPI_Allgather Example

A .

```
// Gather all partial averages down to all the processes
float *sub_avgs = (float *)malloc(sizeof(float) * world_size);
MPI_Allgather(&sub_avg, 1, MPI_FLOAT, sub_avgs, 1, MPI_FLOAT,
              MPI_COMM_WORLD);

// Compute the total average of all numbers.
float avg = compute_avg(sub_avgs, world_size);

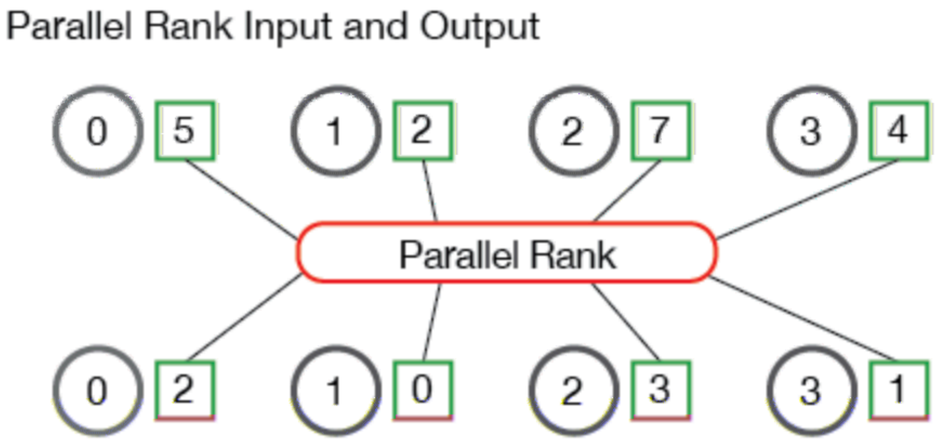
printf("Avg of all elements from proc %d is %f\n", world_rank, avg);
```

```
Avg of all elements from proc 1 is 0.479736
Avg of all elements from proc 3 is 0.479736
Avg of all elements from proc 0 is 0.479736
Avg of all elements from proc 2 is 0.479736
```

https://mpitutorial.com/tutorials/mpi-scatter-gather-and-allgather/

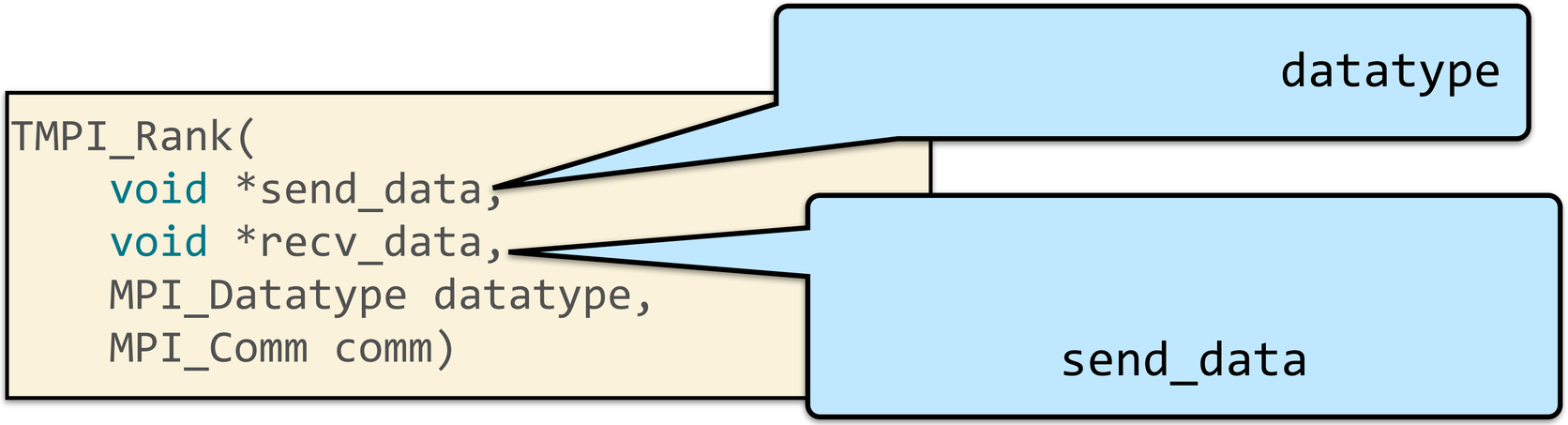
E :

- Input: All processes have **a single number stored in their local memory**
- Output: The **order of the processes' numbers** with respect to the entire set of numbers over all processes.
- **Example application:** benchmarking processors in an MPI cluster and trying to find the order of how fast each processor is relative to the others.



https://mpitutorial.com/tutorials/performing-parallel-rank-with-mpi/

A I



- Input: One number per processor
- Output: Return the associated rank with respect to all numbers across all processors

https://mpitutorial.com/tutorials/performing-parallel-rank-with-mpi/

1: G

```
void *gather_numbers_to_root(void *number, MPI_Datatype datatype,
                             MPI_Comm comm) {
  int comm_rank, comm_size;
  MPI_Comm_rank(comm, &comm_rank);
  MPI_Comm_size(comm, &comm_size);

  int datatype_size;
  MPI_Type_size(datatype, &datatype_size);
  void *gathered_numbers;
  if (comm_rank == 0) {
    gathered_numbers = malloc(datatype_size * comm_size);
  }

  // Gather all of the numbers on the root process
  MPI_Gather(number, 1, datatype, gathered_numbers, 1,
             datatype, 0, comm);

  return gathered_numbers;
}
```

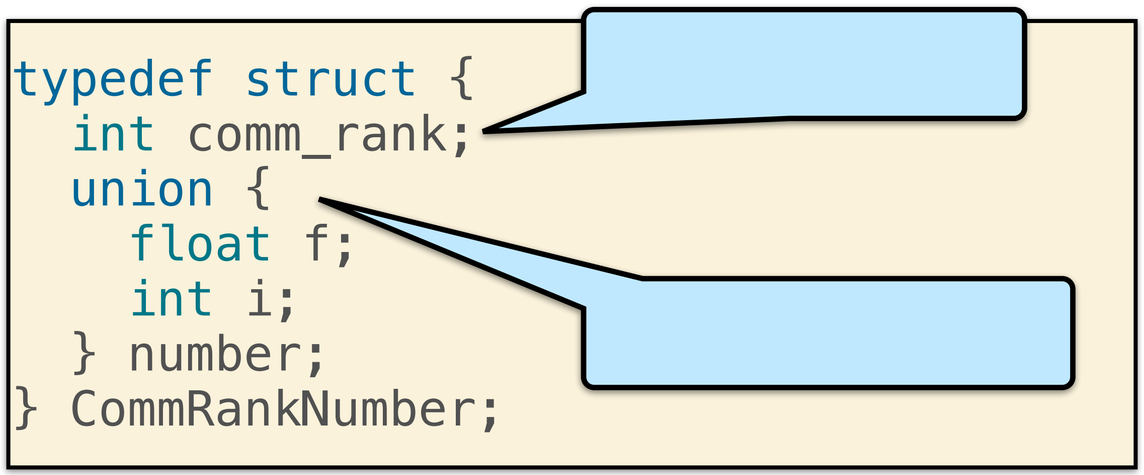
A

G

2:

- Sorting is easily available (e.g., in the C++ STL), but we also have to maintain the ranks that sent the numbers to the root process.

- To attach the owning process to the numbers, we create a struct to hold the information:



2: G

```
int *get_ranks(void *gathered_numbers, int gathered_number_count,
               MPI_Datatype datatype) {
  int datatype_size;
  MPI_Type_size(datatype, &datatype_size);

  // Convert the gathered number array to an array of CommRankNumbers.
  // This allows us to sort the numbers and also keep the information
  // of the processes that own the numbers intact.
  CommRankNumber *comm_rank_numbers = malloc(
    gathered_number_count * sizeof(CommRankNumber));
  int i;
  for (i = 0; i < gathered_number_count; i++) {
    comm_rank_numbers[i].comm_rank = i;
    memcpy(&(comm_rank_numbers[i].number),
           gathered_numbers + (i * datatype_size),
           datatype_size);
  }
…
```

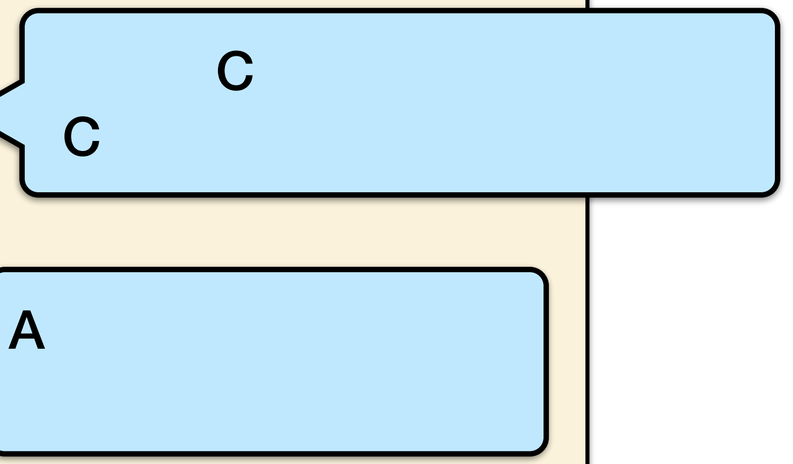
C

C

2: G

```
int *get_ranks(void *gathered_numbers, int gathered_number_count,
               MPI_Datatype datatype) {
  int datatype_size;
  MPI_Type_size(datatype, &datatype_size);

  // Convert the gathered number array to an array of CommRankNumbers.
  // This allows us to sort the numbers and also keep the information
  // of the processes that own the numbers intact.
  CommRankNumber *comm_rank_numbers = malloc(
    gathered_number_count * sizeof(CommRankNumber));
  int i;
  for (i = 0; i < gathered_number_count; i++) {
    comm_rank_numbers[i].comm_rank = i;
    memcpy(&(comm_rank_numbers[i].number),
           gathered_numbers + (i * datatype_size),
           datatype_size);
  }
…
```



2: G

```
int *get_ranks(void *gathered_numbers, int gathered_number_count,
               MPI_Datatype datatype) {
  int datatype_size;
  MPI_Type_size(datatype, &datatype_size);

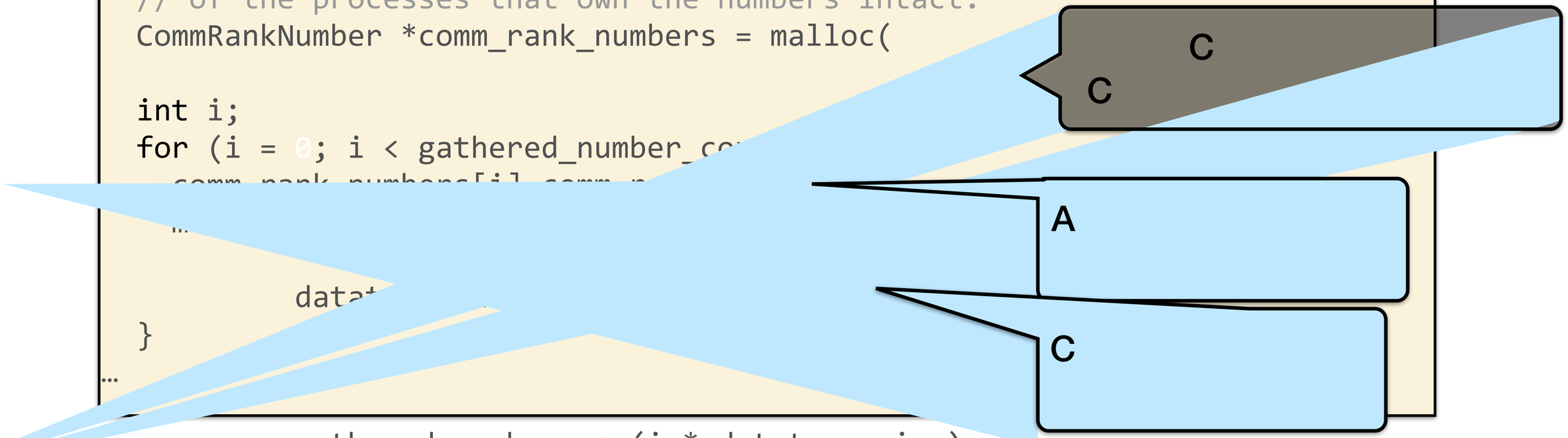
  // Convert the gathered number array to an array of CommRankNumbers.
  // This allows us to sort the numbers and also keep the information
  // of the processes that own the numbers intact.
  CommRankNumber *comm_rank_numbers = malloc(

  int i;
  for (i = 0; i < gathered_number_co
    comm_rank_numbers[i].comm_r
    ...

            datat

  }
...

            gathered_numbers + (i * datatype_size),
```



2: G

```
  if (datatype == MPI_FLOAT) {
    qsort(comm_rank_numbers, gathered_number_count,
          sizeof(CommRankNumber), &compare_float_comm_rank_number);
  } else {
    qsort(comm_rank_numbers, gathered_number_count,
          sizeof(CommRankNumber), &compare_int_comm_rank_number);
  }

  int *ranks = (int *)malloc(sizeof(int) * gathered_number_count);
  for (i = 0; i < gathered_number_count; i++) {
    ranks[comm_rank_numbers[i].comm_rank] = i;
  }

  free(comm_rank_numbers);
  return ranks;
}
```

2: G

```
  if (datatype == MPI_FLOAT) {
    qsort(comm_rank_numbers, gathered_number_count,
          sizeof(CommRankNumber), &compare_float_comm_rank_number);
  } else {
    qsort(comm_rank_numbers, gathered_number_count,
          sizeof(CommRankNumber), &compare_int_comm_rank_n
  }

  int *ranks = (int *)malloc(sizeof(int) * gathered_number_count);
  for (i = 0; i < gathered_number_count; i++) {
    ranks[comm_rank_numbers[i].comm_rank] = i;
  }

  free(comm_rank_numbers);
  return ranks;
}
```

2: G

```
  if (datatype == MPI_FLOAT) {
    qsort(comm_rank_numbers, gathered_number_count,
          sizeof(CommRankNumber), &compare_float_comm_rank_number);
  } else {
    qsort(comm_rank_numbers, gathered_number_count,
          sizeof(CommRankNumber), &compare_int_comm_rank_n
  }

  int *ranks = (int *)malloc(sizeof(int) * gathered_number_count);
  for (i = 0; i < gathered_number_count; i++) {
    ranks[comm_rank_numbers[i].comm_rank] = i;
  }

  free(comm_rank_numbers);
  return ranks;
}
```

-

2: G

```
  if (datatype == MPI_FLOAT) {
    qsort(comm_rank_numbers, gathered_number_count,
          sizeof(CommRankNumber), &compare_float_comm_rank_number);
  } else {
    qsort(comm_rank_numbers, gathered_number_count,
          sizeof(CommRankNumber), &compare_int_comm_rank_n
  }

  int *ranks = (int *)malloc(sizeof(int) * gathered_number_count);
  for (i = 0; i < gathered_number_count; i++) {
    ranks[comm_rank_numbers[i].comm_rank] = i;
  }

  free(comm_rank_numbers);
  return ranks;
}
```

C

-

```
// Gets the rank of the send_data, which is of type datatype. The rank
// is returned in recv_data and is of type datatype.
int TMPI_Rank(void *send_data, void *recv_data, MPI_Datatype datatype,
              MPI_Comm comm) {
  // NOT SHOWN - INIT

  void *gathered_numbers = gather_numbers_to_root(send_data, datatype,
                                                  comm);

  // Get the ranks of each process
  int *ranks = NULL;
  if (comm_rank == 0) {
    ranks = get_ranks(gathered_numbers, comm_size, datatype);
  }

  // Scatter the rank results
  MPI_Scatter(ranks, 1, MPI_INT, recv_data, 1, MPI_INT, 0, comm);

  // NOT SHOWN - CLEANUP
}
```



```
// Gets the rank of the send_data, which is of type datatype. The rank
// is returned in recv_data and is of type datatype.
int TMPI_Rank(void *send_data, void *recv_data, MPI_Datatype datatype,
              MPI_Comm comm) {
  // NOT SHOWN - INIT

  void *gathered_numbers = gather_numbers_to_root(send_data, datatype,
                                                  comm);

  // Get the ranks of each process
  int *ranks = NULL;
  if (comm_rank == 0) {
    ranks = get_ranks(gathered_numbers, comm_size, datatype);
  }

  // Scatter the rank results
  MPI_Scatter(ranks, 1, MPI_INT, recv_data, 1, MPI_INT, 0, comm);

  // NOT SHOWN - CLEANUP
}
```

G

```
// Gets the rank of the send_data, which is of type datatype. The rank
// is returned in recv_data and is of type datatype.
int TMPI_Rank(void *send_data, void *recv_data, MPI_Datatype datatype,
              MPI_Comm comm) {
  // NOT SHOWN - INIT

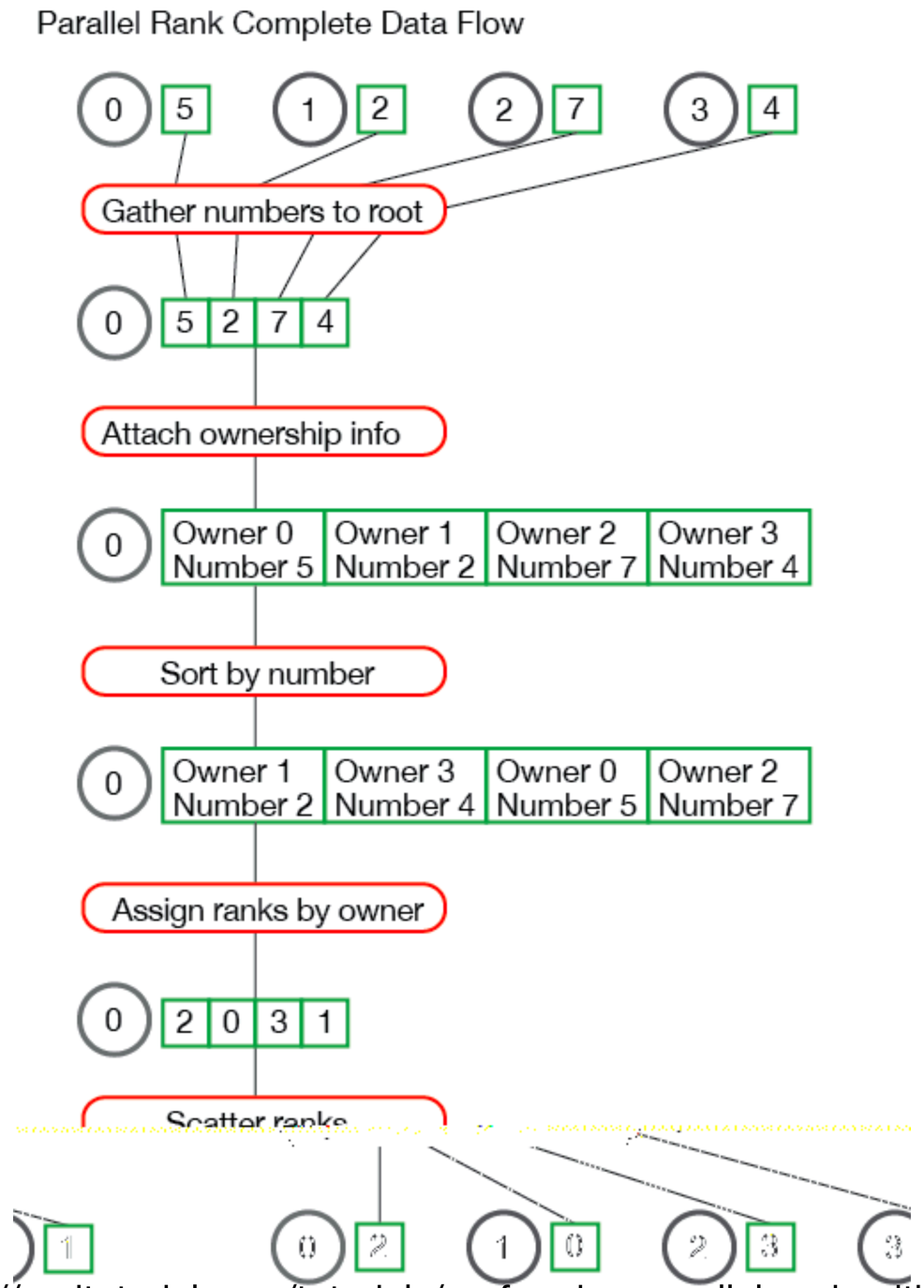
  void *gathered_numbers = gather_numbers_to_root(send_data, datatype,
                                                  comm);

  // Get the ranks of each process
  int *ranks = NULL;
  if (comm_rank == 0) {
    ranks = get_ranks(gathered_numbers, comm_size, datatype);
  }

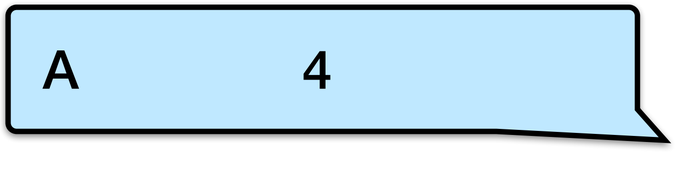
  // Scatter the rank
  MPI_Scatter(ranks, 1, MPI_INT, recv_data, 1, MPI_INT, 0, comm);

  // NOT SHOWN - CLEANUP
}
```

G



https://mpitutorial.com/tutorials/performing-parallel-rank-with-mpi/




```
Rank for 0.242578 on process 0 - 0
Rank for 0.894732 on process 1 - 3
Rank for 0.789463 on process 2 - 2
Rank for 0.684195 on process 3 - 1
```